DUBURI IVC

পানির নিচে দুই ROV-এর মধ্যে শব্দ দিয়ে যোগাযোগ

Acoustic Inter-Vehicle Communication: পুরো গাইড, A থেকে Z.
কেন এভাবে করা হলো, সব formula, circuit, speaker গরম হওয়ার কারণ ও সমাধান, piezo hydrophone seal করার নিয়ম, pool-এর noise filtering, ESP32-এর পুরো code আর তার ব্যাখ্যা, test করার নিয়ম আর সমস্যা হলে কী করবে।

Hardware: ESP32 + PAM8403 class-D amp + car speaker (TX) | piezo disc + op-amp preamp (RX)
Firmware: Arduino-ESP32 core 3.x (3.3.12-এ compile করে দেখা)
সব code: `github.com/jarifalamclaude-max/DUBURI_IVC`

সূচিপত্র

1 এক নজরে পুরো সিস্টেম
2 পানিতে শব্দ: যা জানা দরকার
3 সিস্টেমের block diagram
4 কী কী লাগবে (BOM ও আনুমানিক দাম)
5 Power supply ও grounding
6 Transmitter circuit (ESP32 → amp → speaker)
7 Speaker গরম হচ্ছে কেন, আর সমাধান
8 Speaker box: পানিতে শব্দ বের করা
9 Piezo hydrophone: বানানো ও seal করা
10 Receiver preamp circuit
11 Noise filtering: তিন স্তর
12 Signal design: tone, slot, hopping
13 Packet, CRC, error correction
14 Receiver কীভাবে decode করে (ধাপে ধাপে)
15 Firmware: install, upload, ব্যবহার
16 Code-এর ব্যাখ্যা (function ধরে ধরে)
17 Verification: কী test করা হয়েছে
18 বানানো ও test করার ধাপ
19 Troubleshooting
20 Safety
A সব formula এক জায়গায়
B শব্দকোষ (Glossary)
C পুরো source code

1. এক নজরে পুরো সিস্টেম

একটা ROV (sender) একটা ছোট "packet" পাঠায়। Packet মানে কয়েকটা byte: কাকে পাঠাচ্ছি, কে পাঠাচ্ছে, কোন command (যেমন "LED জ্বালাও"), আর command-এর একটা argument। ESP32 এই byte-গুলোকে কিছু নির্দিষ্ট frequency-র শব্দ (tone)-এ রূপান্তর করে। Car speaker সেই শব্দ পানিতে ছাড়ে। অন্য ROV-এর piezo hydrophone শব্দটা শোনে, preamp signal বড় ও পরিষ্কার করে, ESP32-এর ADC সেটা digital করে। তারপর firmware বের করে কোন tone বেজেছিল, সেখান থেকে byte ফিরিয়ে আনে, CRC দিয়ে যাচাই করে, তারপর কাজটা করে (LED জ্বলে)। চাইলে উল্টো দিকে একটা ছোট "ACK" (পেয়েছি) packet ফেরত পাঠায়।

বিষয়	মান	কেন
Modulation	4-FSK + frequency hopping	সহজ, amplitude-এর উপর নির্ভর করে না, echo-র মধ্যেও কাজ করে
Tone	24টা, 3000–8750 Hz, 250 Hz পরপর	car speaker ও piezo এই band-এ ভালো কাজ করে; pump noise (নিচে) আর RoboSub pinger (25–40 kHz) এড়ায়
Slot	20 ms, প্রতি slot-এ একটা tone = 2 bit	যথেষ্ট লম্বা যাতে echo-র মধ্যেও tone চেনা যায়
Packet দৈর্ঘ্য	64 slot = 1.28 s (preamble 0.2 s + data 1.08 s)	command পাঠানোর জন্য যথেষ্ট দ্রুত
Payload	4 byte (dst, src, seq+flags, cmd, arg) + CRC-16	"LED on"-এর মতো command-এর জন্য যথেষ্ট
Error correction	Convolutional code K=7, rate 1/2 + Viterbi	যে bit ভুল আসে সেগুলো ঠিক করে ফেলে
Receiver ADC	ESP32 ADC1, DMA, 50 kHz	কোনো CPU সময় নষ্ট না করে sample নেয়
দূরত্ব (লক্ষ্য)	10–25 m, RoboSub-এর মতো pool	—
Simulation-এ ফল	in-band SNR −6 dB-এ 90%, 0 dB-এ 100% packet ঠিক	অধ্যায় 17

Speaker গরম হওয়ার সংক্ষিপ্ত উত্তর (বিস্তারিত অধ্যায় 7): transistor বা L298N দিয়ে speaker-এর এক মাথা GND-এ রেখে on/off করলে অর্ধেক power DC হয়ে যায়। এতে শব্দ হয় না, শুধু coil গরম হয়। সমাধান হলো BTL class-D amplifier (PAM8403), amp-এর input-এ filter আর volume control, আর firmware-এর "heat guard" যা speaker-কে কাজের পর বিশ্রাম দেয়।

Piezo seal করলে শব্দ ধরবে কি না (বিস্তারিত অধ্যায় 9): ধরবে। Epoxy, silicone, তেল, এদের "acoustic impedance" পানির কাছাকাছি, তাই শব্দ প্রায় পুরোটাই ভেতরে ঢোকে (৮৫–৯৯%)। শব্দ আটকায় শুধু **বাতাস** (০.১%-এর মতো ঢোকে)। তাই নিয়ম একটাই: piezo আর পানির মাঝে কোনো air bubble বা ফাঁকা জায়গা থাকবে না।

2. পানিতে শব্দ: যা জানা দরকার

2.1 Radio কেন চলে না, শব্দ কেন চলে

পানি, বিশেষ করে chlorine দেওয়া pool-এর পানি, বিদ্যুৎ পরিবাহী। তাই WiFi বা Bluetooth-এর মতো radio wave কয়েক সেন্টিমিটারের মধ্যেই শেষ হয়ে যায়। শব্দ উল্টো: পানিতে বাতাসের চেয়ে প্রায় ৪.৫ গুণ দ্রুত যায় আর অনেক কম ক্ষয় হয়। তাই পানির নিচে যোগাযোগের প্রধান উপায় হলো শব্দ (acoustic)।

```
c ≈ 1480 – 1500 m/s   (পানিতে শব্দের গতি; বাতাসে 343 m/s)
λ = c / f
```

λ = wavelength। উদাহরণ: f = 5000 Hz হলে λ = 1480/5000 ≈ 0.30 m। এই সংখ্যাটা পরে লাগবে। যেকোনো জিনিস λ-এর তুলনায় অনেক পাতলা হলে (যেমন 2–3 mm epoxy) শব্দ সেটা প্রায় টেরই পায় না।

```
t = d / c
```

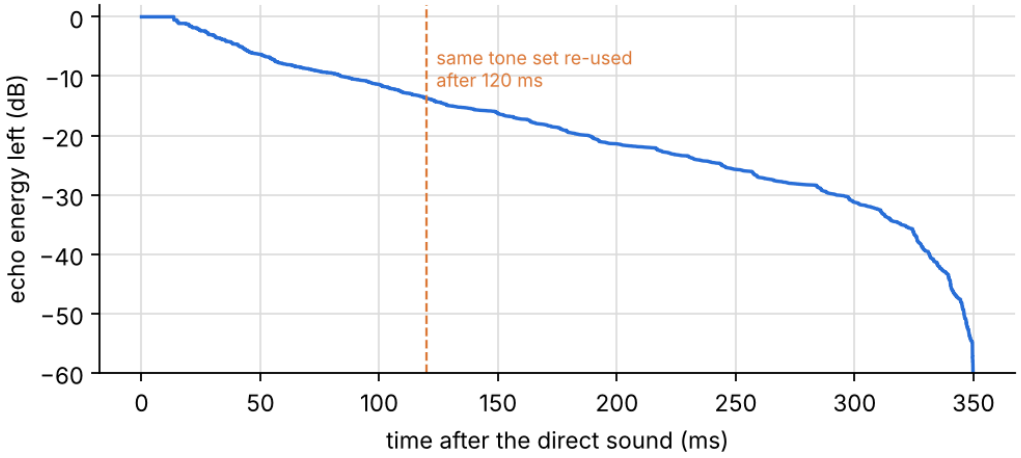
শব্দ পৌঁছাতে কত সময় লাগে। 25 m হলে t = 25/1480 ≈ 17 ms। এটা একটা slot (20 ms)-এর চেয়ে কম, কিন্তু pool-এর দেয়ালে ধাক্কা খেয়ে আসা echo আসে আরো অনেক পরে (৫০–৩০০ ms পর্যন্ত)।

2.2 Pool-এর সবচেয়ে বড় সমস্যা: echo (reverberation)

Pool-এর দেয়াল আর মেঝে টাইলস বা কংক্রিটের, আর পানির উপরিতল প্রায় আয়নার মতো শব্দ ফেরত দেয়। তাই একটা tone বাজালে receiver সেটা একবার না, শত শত বার শোনে, প্রতিটা আলাদা দেরিতে আর আলাদা phase-এ। দুটো খারাপ ফল হয়:

1. **Fading:** কোনো জায়গায় কোনো একটা frequency-র echo-গুলো একে অপরকে কেটে দেয়, ফলে সেই tone খুব দুর্বল শোনায়। অন্য frequency আবার ঠিক থাকে।
2. **Inter-symbol interference:** আগের slot-এর tone-এর echo পরের slot-এও বাজতে থাকে।

আমাদের design-এর প্রায় প্রতিটা সিদ্ধান্ত এই দুটো সমস্যা ঠেকানোর জন্য: frequency hopping (অধ্যায় 12), interleaving আর convolutional code (অধ্যায় 13)। নিচের চিত্রে simulation-এর pool model-এ echo-র শক্তি কত দ্রুত কমে, সেটা দেখানো হয়েছে।



চিত্র 2.1: 50×25×5 m pool model-এ echo-র অবশিষ্ট শক্তি। একই frequency set আবার ব্যবহার করার আগে 120 ms অপেক্ষা করা হয়, ততক্ষণে echo অনেকটা কমে যায়।

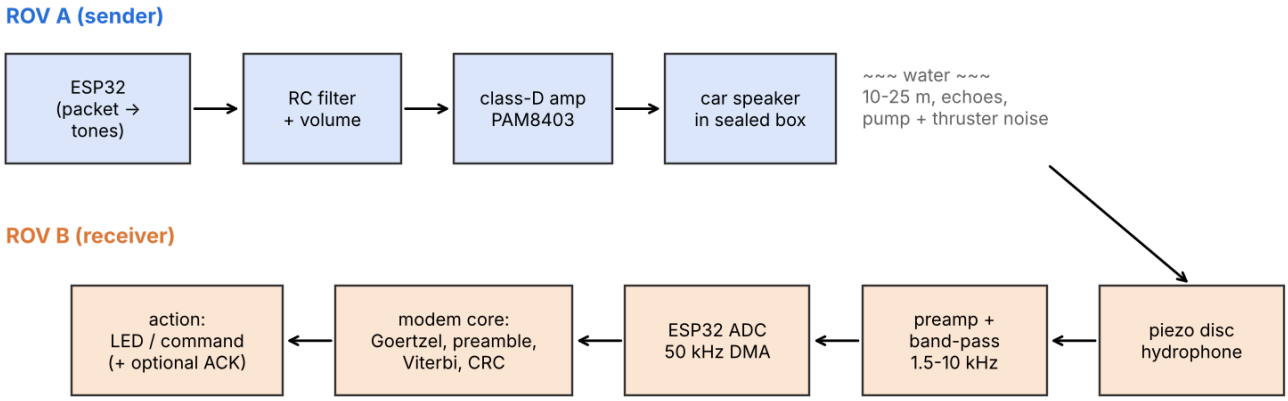
2.3 RoboSub-এ বিশেষ সতর্কতা

RoboSub-এ acoustic pinger থাকে, যার frequency সাধারণত 25–40 kHz। আমাদের tone 3–8.75 kHz, অর্থাৎ অনেক নিচে। তার উপর ADC-র sampling rate 50 kHz নেওয়া হয়েছে যাতে pinger-এর শব্দ "alias" হয়ে আমাদের band-এ ঢুকে না পড়ে (অধ্যায় 11.2-এ formula)। Thruster আর pool pump-এর বেশিরভাগ শব্দ 2–3 kHz-এর নিচে থাকে, তাই আমাদের band সেটাও এড়িয়ে যায়।

2.4 Doppler: ROV চললে frequency সরে কি?

> $\Delta f = f \cdot v / c$
>
> v = দুই ROV-এর আপেক্ষিক গতি। v = 0.5 m/s আর f = 8750 Hz হলে Δf = 8750 × 0.5 / 1480 ≈ 3 Hz। আমাদের receiver-এর frequency resolution 62.5 Hz আর tone 250 Hz পরপর, তাই 3 Hz কোনো সমস্যা না।

3. সিস্টেমের block diagram



চিত্র 3.1: উপরে sender, নিচে receiver। দুই ROV-এ একই hardware থাকলে দুজনেই পাঠাতে ও শুনতে পারে (ACK-এর জন্য)।

Sender-এর কাজ: packet → bit → error-correction code → interleave → প্রতি 2 bit-এ একটা tone → ESP32-এর LEDC peripheral দিয়ে square wave → RC filter দিয়ে মসৃণ করা ও volume ঠিক করা → PAM8403 amp → speaker → box-এর পাতলা জানালা → পানি।

Receiver-এর কাজ: piezo → preamp (gain + 1.5–10 kHz band-pass) → ESP32 ADC (DMA, 50 kHz) → DC সরানো → Goertzel দিয়ে 24টা tone-এর শক্তি মাপা → noise floor দিয়ে ভাগ (whitening) → preamble খোঁজা → timing ঠিক করা → প্রতি slot-এর soft bit → de-interleave → Viterbi → CRC check → command চালানো।

একটা গুরুত্বপূর্ণ design সিদ্ধান্ত: modem-এর মূল code (`am_core.c`) সাধারণ C-তে লেখা, এতে ESP32-এর কিছু নেই। তাই একই file PC-তে compile করে test আর simulation চালানো গেছে। PC-তে যেটা test হয়েছে, ঠিক সেটাই ROV-এ চলে।

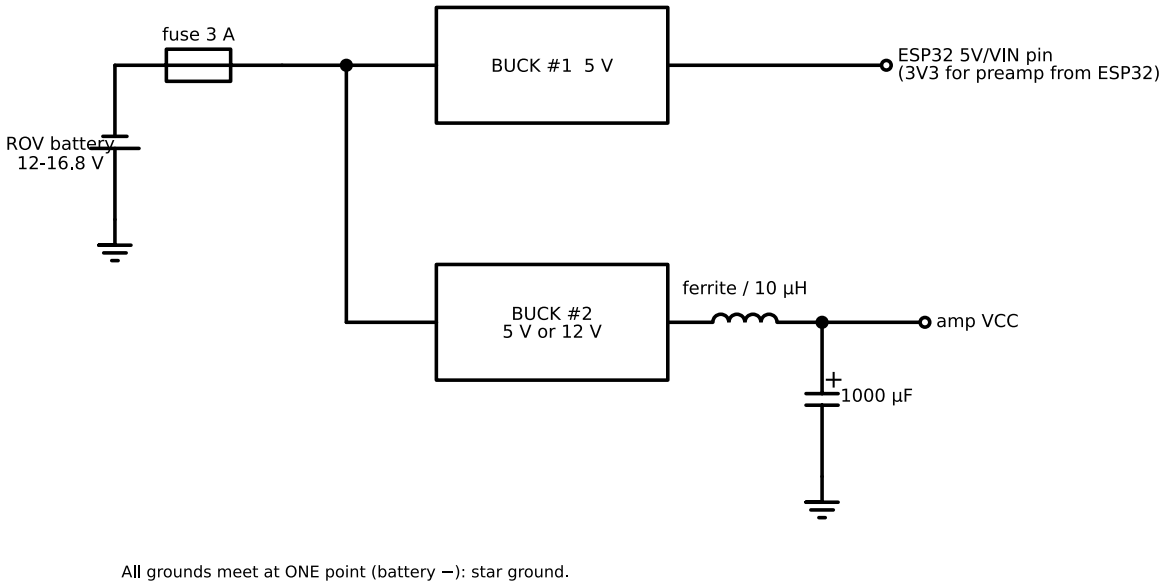
4. কী কী লাগবে (BOM ও আনুমানিক দাম)

প্রতিটা ROV-এর জন্য (দুটো ROV-এ দুই সেট)। দাম বাংলাদেশের সাধারণ অনলাইন বা দোকানের **আনুমানিক** দাম, দোকানভেদে কমবেশি হবে।

জিনিস	সংখ্যা	আনুমানিক দাম (৳)	মন্তব্য
ESP32 DevKit V1 (WROOM-32)	1	450–650	Uno-র চেয়ে সস্তা, অনেক দ্রুত, DMA ADC আছে
PAM8403 class-D amp module	1	60–120	5 V, 3 W×2, BTL output
(বিকল্প) PAM8610 amp module	(1)	250–400	12 V, বেশি জোর লাগলে; heat sink লাগবে
Car speaker (আছে)	1	—	4 Ω
Piezo disc 27 mm বা 35 mm	2–3	10–20 প্রতিটা	একটা নষ্ট হলে বদলানোর জন্য বাড়তি রাখো
MCP6002 op-amp (না পেলে LM358)	1	40–80 (LM358: 10–15)	dual op-amp, DIP-8
Buck converter (LM2596 বা MP1584)	2	80–150 প্রতিটা	একটা ESP32-এর, একটা amp-এর
Trimmer pot 10 kΩ, 100 kΩ	1 + 1	10–20 প্রতিটা	volume ও gain
Resistor, capacitor (তালিকা অধ্যায় 6, 10)	—	100–150	film/ceramic cap
Shielded cable (2-core বা mic cable)	1–2 m	30–60/m	hydrophone-এর জন্য
2-part epoxy বা liquid electrical tape	1	150–300	piezo seal
Acrylic sheet 3–5 mm (speaker জানালা)	ছোট টুকরা	100–200	অধ্যায় 8
Perfboard, header, fuse, তার	—	100–150	—

কেন Uno না: Uno-র RAM 2 KB, clock 16 MHz, ADC দ্রুত হলে noisy হয়, আর DMA নেই। ESP32-এ 520 KB RAM, 240 MHz, float FPU আর DMA দিয়ে 50 kHz ADC sampling আছে, আর দামও Uno clone-এর মতো বা কম। আমাদের receiver প্রতি সেকেন্ডে প্রায় ৮০ লাখ গুণ-যোগ করে, যা Uno-তে সম্ভব না।

5. Power supply ও grounding



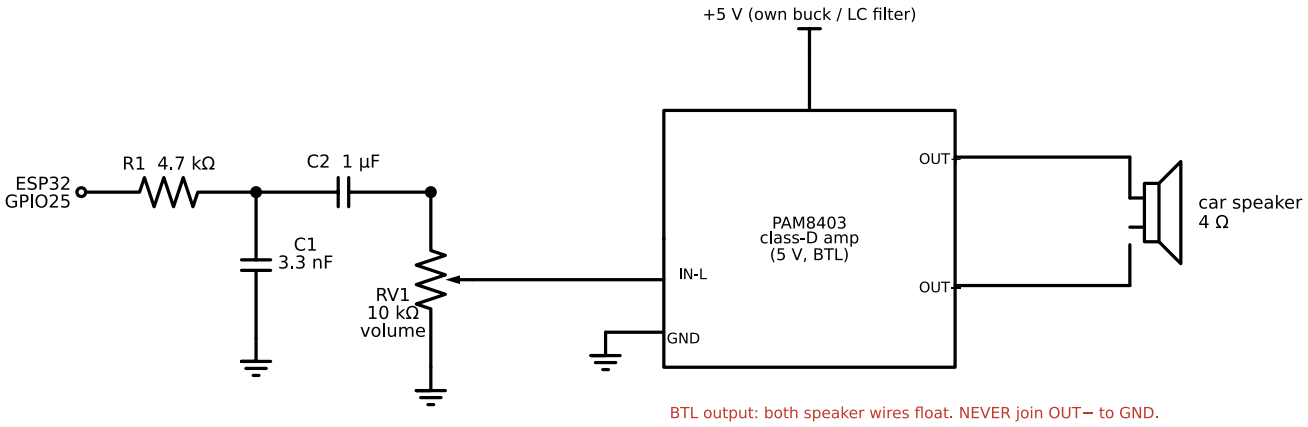
চিত্র 5.1: দুটো আলাদা buck converter। Amp-এর supply-তে ferrite/inductor আর 1000 µF।

- **দুটো buck কেন:** amp প্রতিবার tone বাজানোর সময় হঠাৎ ১ A-এর মতো current টানে। একই 5 V থেকে ESP32 চললে voltage নেমে ESP32 reset হতে পারে (brownout), আর preamp-এ শব্দ (noise) ঢোকে।
- **Ferrite + 1000 µF:** একটা LC low-pass filter। Amp-এর switching noise buck-এ ফিরতে দেয় না, আর হঠাৎ current-এর সময় capacitor থেকে charge যোগায়।
- **Star ground:** সব GND তার একটা বিন্দুতে (battery −) মিলবে। Thruster-এর বড় current যেন preamp-এর GND তারের মধ্য দিয়ে না যায়।
- **Preamp-এর supply:** ESP32-এর 3V3 থেকে 10 Ω + 100 µF দিয়ে আলাদা করে "3V3A" (অধ্যায় 10)।

> $I_{battery} \approx P_{out} / (\eta \cdot V_{battery})$
>
> Buck converter-এর input current। η = efficiency (≈ 0.85)। উদাহরণ: amp 5 V-এ 1 A টানলে (5 W), 14.8 V battery থেকে লাগে 5 / (0.85 × 14.8) ≈ 0.4 A। তাই 3 A fuse যথেষ্ট।

> $f_{LC} = 1 / (2\pi \sqrt{(L \cdot C)})$
>
> 10 µH আর 1000 µF হলে f ≈ 1.6 kHz। এর উপরের buck switching noise (সাধারণত 150 kHz+) অনেকটা কেটে যায়।

6. Transmitter circuit (ESP32 → amp → speaker)



চিত্র 6.1: Transmitter। ESP32-এর GPIO25 থেকে square wave → RC filter → volume → PAM8403 → speaker।

6.1 প্রতিটা অংশ কী করে

- **GPIO25 (LEDC):** ESP32-এর LEDC peripheral hardware দিয়েই নির্দিষ্ট frequency-র square wave বানায় (0 V আর 3.3 V-এর মধ্যে)। CPU কিছু করে না, তাই tone একদম স্থির থাকে। প্রতি 20 ms পরপর firmware শুধু frequency বদলে দেয়।
- **R1 + C1 (low-pass filter):** square wave-এর ধারালো কিনারা কেটে নরম করে, উঁচু harmonic কমায়।
- **C2 (1 µF):** DC আটকায়। Amp শুধু AC পায়, এটা heat কমানোর জন্যও জরুরি (অধ্যায় 7)।
- **RV1 (10 kΩ trimmer):** volume। ESP32-এর 2.1 V peak signal amp-এর জন্য অনেক বেশি, ফলে amp clip করত। Clip মানে output square wave, যা বেশি গরম করে।
- **PAM8403:** class-D, BTL (bridge-tied load) amp, ~৯০% efficient। নিজে প্রায় গরম হয় না।

6.2 Formula ও হিসাব

$$\text{square wave } (0\ldots V) = V/2 + (2V/\pi)\cdot[\sin \omega t + (1/3)\sin 3\omega t + (1/5)\sin 5\omega t + \ldots]$$

Fourier series। প্রথম অংশ V/2 হলো DC। আসল tone (fundamental)-এর amplitude $2V/\pi$; ESP32-এ V = 3.3 V হলে 2.10 V peak। শুধু বিজোড় harmonic (3f, 5f, ...) থাকে। আমাদের সবচেয়ে নিচের tone 3000 Hz-এর 3f = 9000 Hz, যা সবচেয়ে উঁচু tone 8750 Hz-এর উপরে। তাই harmonic কখনো অন্য tone-এর জায়গায় পড়ে না।

$$f_c = 1 / (2\pi \cdot R_{th} \cdot C1),\ R_{th} = R1 \parallel RV1 = (4.7k \times 10k)/(4.7k + 10k) = 3.2\ k\Omega$$

$$f_c = 1 / (2\pi \times 3.2\ k\Omega \times 3.3\ nF) \approx 15\ kHz$$

Low-pass corner। 15 kHz-এর নিচের (আমাদের tone) প্রায় পুরোটা যায়, উপরের harmonic কমে।

$$\text{gain_divider} = RV1 / (R1 + RV1) = 10 / 14.7 = 0.68 \rightarrow RV1\text{-এর মাথায় } 2.10 \times 0.68 \approx 1.43\ V \text{ peak}$$

$$f_{HP} = 1 / (2\pi\ (R1 + RV1)\ C2) = 1 / (2\pi \times 14.7\ k\Omega \times 1\ \mu F) \approx 11\ Hz$$

C2 শুধু DC আটকায়। 11 Hz-এর উপরের সব কিছু (আমাদের tone) সহজে যায়।

```
PAM8403 gain ≈ 24 dB = ×15.8, BTL max peak ≈ V_supply = 5 V
P_sine = V_peak² / (2 R) = 5² / (2 × 4) ≈ 3.1 W (clip হওয়ার ঠিক আগে)
V_in দরকার = V_out / gain = 4 V / 15.8 ≈ 0.25 V peak → RV1 প্রায় 18% ঘুরানো
```

তাই RV1 একদম নিচ থেকে শুরু করবে, আস্তে আস্তে বাড়াবে। যেখানে শব্দ আর বাড়ে না, বরং কর্কশ হয়, সেখানে clip শুরু; তার একটু নিচে রাখবে। 4 V peak হলে $4^2/(2\times4) = 2$ W।

BTL সতর্কতা: PAM8403/PAM8610-এর output-এর দুই পাশেই signal থাকে (একটা উঠলে অন্যটা নামে)। Speaker-এর কোনো তার **কখনো GND-এ লাগাবে না**, আর দুটো channel-এর OUT− একসাথে জোড়া দেবে না। দিলে amp-এর output short হয়ে amp গরম হবে বা পুড়ে যাবে। Speaker-এর দুই তার শুধু L+ আর L−-এ যাবে।

6.3 বেশি জোর লাগলে: PAM8610

PAM8403 যথেষ্ট না হলে (scope-এ q < 10 dB, অধ্যায় 18) PAM8610 module 12 V buck থেকে চালাও। এতে 12 V BTL, অর্থাৎ max ≈ $12^2/(2\times4) = 18$ W (তাত্ত্বিক)। কিন্তু মনে রাখো, power বাড়লে heat-ও বাড়ে। প্রথমে gain বাড়াও, transducer coupling ঠিক করো (অধ্যায় 8), তারপর power। PAM8610-এ heat sink লাগাবে। Module-এ MUTE/SD pin থাকলে `PIN_AMP_ENABLE` দিয়ে ESP32 শুধু পাঠানোর সময় amp চালু রাখবে।

7. Speaker গরম হচ্ছে কেন, আর সমাধান

Speaker-এর voice coil আসলে একটা তামার তারের কুণ্ডলী, যার resistance R (car speaker-এ ~3.2–4 Ω)। Coil-এ যত electrical power যায়, তার প্রায় পুরোটাই (৯৯%-এর বেশি) তাপ হয়। শব্দ হয় খুব সামান্য (speaker-এর efficiency ~১%)। তাই প্রশ্ন হলো, coil-এ অদরকারি power কোথা থেকে আসছে।

7.1 কারণ 1 (সবচেয়ে সাধারণ): DC current

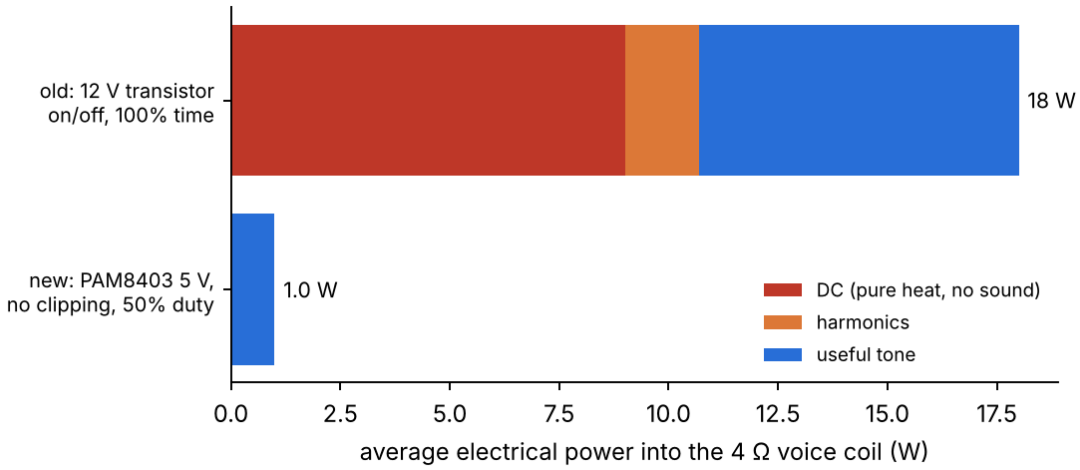
অনেকে speaker-এর এক মাথা GND-এ রেখে transistor/MOSFET বা L298N-এর এক অংশ দিয়ে 0 আর V-এর মধ্যে on/off করে। এই signal-এর গড় মান V/2, মানে coil-এ একটা স্থায়ী DC current চলে। DC-তে cone একপাশে ঠেলে থেমে থাকে, কোনো শব্দ হয় না, কিন্তু I^2R তাপ হয়।

$$P_{total} = d \cdot V^2 / R \quad (d = \text{duty cycle} = 0.5)$$

$$P_{DC} = (d \cdot V)^2 / R \leftarrow \text{শুধু তাপ, কোনো শব্দ নয়}$$

$$P_{tone} = (2V/\pi)^2 / (2R) \leftarrow \text{কাজের অংশ}$$

12 V, 4 Ω হলে P_total = 0.5 × 144 / 4 = 18 W, P_DC = 36/4 = **9 W**, P_tone = 7.3 W, বাকি 1.7 W harmonic। অর্থাৎ অর্ধেক power সরাসরি তাপ। Box বন্ধ থাকলে এই তাপ বের হওয়ারও জায়গা পায় না।



চিত্র 7.1: পুরনো পদ্ধতি বনাম নতুন। নতুন পদ্ধতিতে গড় power প্রায় ১৮ ভাগের এক ভাগ।

7.2 কারণ 2: সারাক্ষণ বাজানো (100% duty)

Tone একটানা বাজালে coil ঠান্ডা হওয়ার সময় পায় না। আমাদের packet মাত্র 1.28 s। Firmware-এর **heat guard** প্রতিবার পাঠানোর পর বাধ্যতামূলক বিশ্রাম দেয়:

$$t_{rest} = t_{on} \cdot (1 - d) / d$$

$$P_{avg} = P_{on} \cdot d$$

d = `MAX_TX_DUTY` (config.h, default 0.5)। 1.28 s packet-এর পর অন্তত 1.28 s চুপ। P_on = 2 W হলে গড় 1 W। Speaker-এর rating সাধারণত 20–50 W, তাই এটা খুবই নিরাপদ।

7.3 কারণ 3: Amp clip করা

Input বেশি হলে amp-এর output square হয়ে যায় (clipping)। একই peak voltage-এ square wave-এর power sine-এর দ্বিগুণ (V^2/R বনাম $V^2/2R$), আর বাড়তি অংশটা harmonic, যা কাজে লাগে না। RV1 দিয়ে clip-এর নিচে রাখো।

7.4 কারণ 4: ভুল driver

L298N-এর ভেতরে transistor-এ 2–3 V voltage drop হয়। 1.5 A-তে সেটা 3–4.5 W, যা শুধু L298N-কে গরম করে। সাথে চলে উপরের DC সমস্যা। Class-D amp-এর efficiency ~৯০%, তাই এই সমস্যা থাকে না।

7.5 কারণ 5: বন্ধ box, তাপ বের হওয়ার পথ নেই

> $\Delta T = P_{avg} \times R_{th}$
>
> R_{th} = thermal resistance (°C/W): প্রতি watt-এ তাপমাত্রা কত বাড়ে। বাতাস ভালো insulator, তাই বন্ধ box-এ R_{th} অনেক বেশি। উদাহরণ (ধরে নেওয়া মান): R_{th} = 10 °C/W হলে পুরনো পদ্ধতির 9 W DC → 90 °C বেশি! নতুন পদ্ধতির 1 W → 10 °C। সমাধান: speaker-এর magnet থেকে একটা aluminium পাত box-এর দেয়াল পর্যন্ত লাগাও (চিত্র 8.1)। দেয়ালের ওপাশে পানি খুব ভালো ঠান্ডা করে।

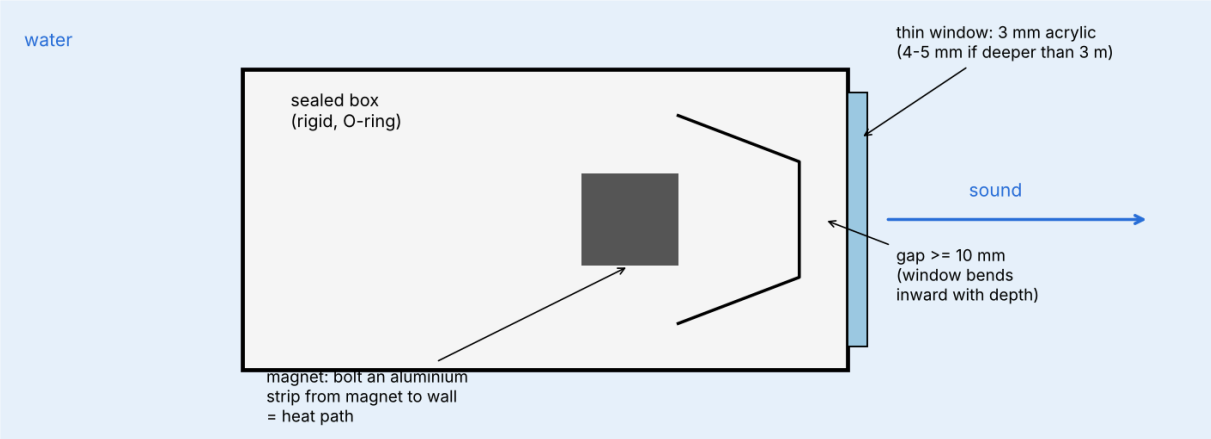
7.6 Coil-এর তাপমাত্রা নিজে মাপার উপায়

> $R(T) = R_0\,(1 + \alpha\,\Delta T),\ \alpha_{copper} = 0.00393\ /°C$
>
> $\Delta T = (R_{hot} / R_{cold} - 1) / 0.00393$
>
> Multimeter দিয়ে speaker-এর resistance ঠান্ডা অবস্থায় মাপো (ধরো 3.4 Ω)। কিছুক্ষণ চালিয়ে amp খুলে সাথে সাথে আবার মাপো (ধরো 4.1 Ω)। তাহলে $\Delta T = (4.1/3.4 - 1)/0.00393 \approx 52$ °C। 60–70 °C-এর বেশি বাড়লে `MAX_TX_DUTY` বা volume কমাও।

Checklist: (1) class-D BTL amp, speaker GND-এ নয়; (2) C2 দিয়ে DC block; (3) RV1-এ clip-এর নিচে; (4) MAX_TX_DUTY 0.5 বা কম; (5) magnet থেকে দেয়ালে তাপের পথ; (6) R মেপে তাপমাত্রা দেখা।

8. Speaker box: পানিতে শব্দ বের করা

Speaker বন্ধ box-এ আছে। Cone নড়লে box-এর ভেতরের বাতাস নড়ে, বাতাস box-এর দেয়াল নাড়ায়, দেয়াল পানি নাড়ায়। বাতাস আর পানির মধ্যে impedance-এর বিশাল পার্থক্য (অধ্যায় 9.1), তাই এখানেই সবচেয়ে বেশি শক্তি হারায়। উপায় হলো দেয়ালের একটা অংশকে হালকা ও পাতলা "জানালা" বানানো, যা নিজে drum-এর মতো কাঁপে।



চিত্র 8.1: Speaker-এর মুখ পাতলা acrylic জানালার দিকে, মাঝে ফাঁক, magnet থেকে দেয়াল পর্যন্ত তাপের পথ।

- Speaker-এর মুখ একটা পাতলা, শক্ত জানালার দিকে রাখো (3 mm acrylic/polycarbonate)। জানালা যত পাতলা আর হালকা, তত ভালো শব্দ যায়, কিন্তু গভীরতায় চাপ সহ্য করতে হবে (নিচের formula)।
- Cone আর জানালার মাঝে অন্তত 10 mm ফাঁক রাখো, কারণ গভীরে জানালা ভেতরে বাঁকে।
- Box-এর বাকি দেয়াল মোটা আর শক্ত রাখো, O-ring দিয়ে seal করো।
- (Advanced) পুরো box তেলে ভরে rubber diaphragm দিলে coupling আর cooling দুটোই অনেক ভালো হয়। কিন্তু এটা বানানো কঠিন, তেল চুঁইয়ে পড়ার ঝুঁকি আছে, আর cone তেলে ভিজে যায়। প্রথম version-এ এটা কোরো না।
- (বিকল্প পরীক্ষা) একটা piezo disc-কেও speaker হিসেবে চালানো যায় (amp-এর output সরাসরি piezo-তে)। 3–6 kHz-এ পানিতে এটা ভালো কাজ করে। Car speaker-এর সাথে তুলনা করতে পারো।

8.1 গভীরতায় চাপ ও জানালার পুরুত্ব

> $P = \rho \cdot g \cdot h$
>
> 5 m গভীরে P = 1000 × 9.81 × 5 ≈ 49 kPa (≈ 0.49 bar)। জানালার প্রতি বর্গ সেন্টিমিটারে প্রায় 0.5 kg-এর ভার।

> $D = E\,t^3 / (12\,(1 - \nu^2))$
>
> $w_{max} = P\,a^4 / (64\,D)$ (কিনারা আটকানো গোল plate-এর মাঝখানে কতটা বাঁকে)
>
> $\sigma_{max} = 3\,P\,a^2 / (4\,t^2)$ (সবচেয়ে বেশি stress, কিনারায়)
>
> a = জানালার ব্যাসার্ধ, t = পুরুত্ব। Acrylic: E ≈ 3 GPa, ν ≈ 0.37, tensile strength ≈ 70 MPa।
>
> উদাহরণ, 5 m গভীরে: 10 cm জানালা (a = 5 cm), t = 3 mm → w ≈ 0.6 mm, σ ≈ 10 MPa (safety factor ≈ 7) ✔।
>
> 15 cm জানালা (a = 7.5 cm), t = 3 mm → w ≈ 3.1 mm, σ ≈ 23 MPa (safety factor ≈ 3, একটু বেশি বাঁকে)। t = 5 mm হলে w ≈ 0.7 mm, σ ≈ 8 MPa ✔। তাই 3 m-এর বেশি গভীরে বড় জানালার জন্য 4–5 mm নাও। এগুলো মোটামুটি হিসাব (ছোট deflection ধরে), পানিতে নামানোর আগে বালতি বা অগভীর পানিতে leak test করবে।

9. Piezo hydrophone: বানানো ও seal করা

9.1 Seal করলে শব্দ ঢুকবে কেন (physics)

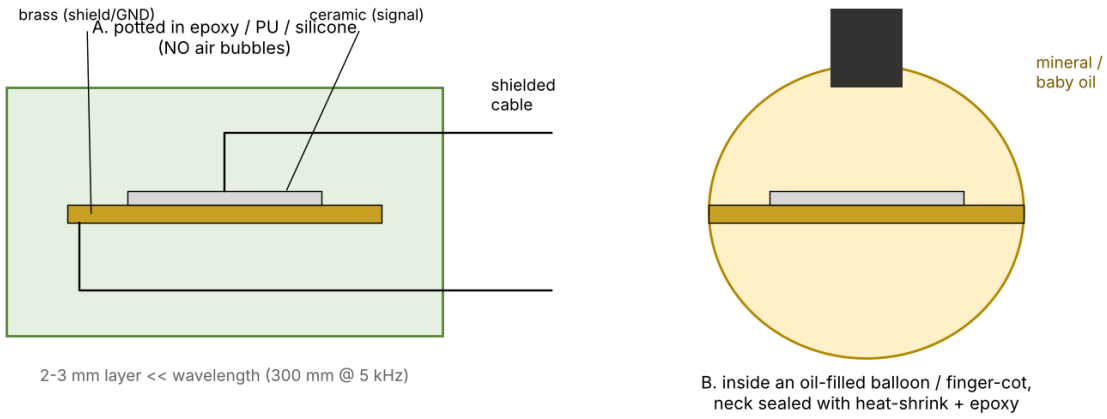
শব্দ হলো চাপের ঢেউ। এক পদার্থ থেকে অন্য পদার্থে ঢোকার সময় কতটা ঢুকবে আর কতটা ফেরত যাবে, সেটা নির্ভর করে দুই পদার্থের **acoustic impedance**-এর উপর:

> $Z = \rho \cdot c$ (ঘনত্ব × শব্দের গতি, একক Rayl)
>
> $T = 4 Z_1 Z_2 / (Z_1 + Z_2)^2$ (কত ভাগ শক্তি ঢোকে)
>
> পানি Z ≈ 1.48 MRayl। Epoxy ≈ 3.0 → T ≈ 0.89 (৮৯% ঢোকে)। Silicone rubber ≈ 1.05 → T ≈ 0.97। Mineral/baby oil ≈ 1.2 → T ≈ 0.99। **বাতাস ≈ 0.0004 → T ≈ 0.001 (০.১%!)**।
>
> তাই কোটিং কোনো সমস্যা না, সমস্যা শুধু বাতাস। আর কোটিং যেহেতু wavelength (5 kHz-এ 300 mm)-এর তুলনায় খুব পাতলা (2–3 mm), শব্দ তার মধ্য দিয়ে প্রায় না থেমে চলে যায়।

9.2 Piezo disc কীভাবে শব্দ ধরে

Piezo disc-এ একটা পিতলের (brass) প্লেটের উপর পাতলা ceramic লাগানো থাকে। চাপ পড়ে ceramic একটু বাঁকলে তার দুই পাশে voltage তৈরি হয় (piezoelectric effect)। পানির চাপের ঢেউ disc-কে বাঁকায়, তাই disc থেকে millivolt মাপের signal আসে। Disc বৈদ্যুতিকভাবে একটা ছোট capacitor-এর মতো (প্রায় 10–30 nF), তাই এর পরে high-impedance input (1 MΩ) লাগে।

9.3 বানানোর ধাপ



চিত্র 9.1: দুটো পদ্ধতি। A: epoxy/silicone-এ ডুবানো (potting)। B: তেলভর্তি balloon/finger-cot।

1. **Cable:** shielded cable (mic cable)। ভেতরের তার ceramic (রুপালি) অংশে, shield/braid brass-এর কিনারায়। Ceramic-এ খুব দ্রুত (২–৩ সেকেন্ড) solder করো, বেশি তাপে রুপালি স্তর উঠে যায়।
2. **পরিষ্কার:** isopropyl alcohol দিয়ে তেল আর flux মুছে ফেলো, শুকাতে দাও।
3. **Strain relief:** cable-কে disc-এর কাছে একটু গিঁট দিয়ে বা heat-shrink দিয়ে আটকে দাও, যাতে টান solder-এ না পড়ে।
4. **পদ্ধতি A (epoxy/silicone potting):** একটা ছোট ছাঁচে (বোতলের ছিপি, PVC end cap) disc দাঁড় করিয়ে রাখো। 2-part epoxy ধীরে ধীরে এক কোণ থেকে ঢালো, যাতে বাতাস বের হয়ে যেতে পারে। Disc-এর দুই পাশে 2–3 mm স্তর রাখো। Bubble দেখলে টুথপিক দিয়ে ফাটিয়ে দাও। হালকা গরম জায়গায় (hair dryer দূর থেকে) রাখলে epoxy পাতলা

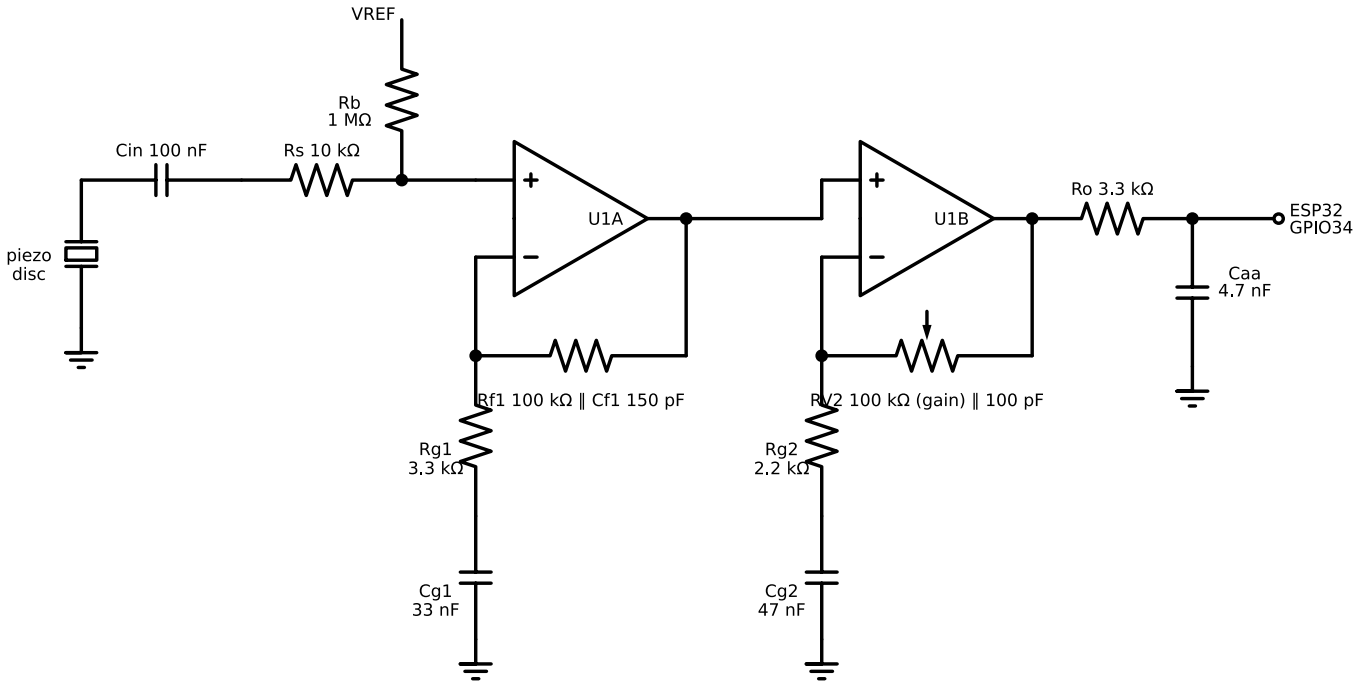
হয় আর bubble সহজে ওঠে। পুরো ২৪ ঘণ্টা শুকাতে দাও।
সস্তা বিকল্প: liquid electrical tape / Plasti Dip-এ ৩–৪ বার ডুবিয়ে পাতলা স্তর, প্রতিবার শুকিয়ে নিয়ে। Hot glue-ও চলে, কিন্তু bubble থাকার সম্ভাবনা বেশি।

5. **পদ্ধতি B (তেল):** finger-cot বা ছোট balloon-এ baby/mineral oil ভরে disc ডোবাও। Bubble বের করে দাও, তারপর মুখটা cable-এর উপর heat-shrink + epoxy দিয়ে বন্ধ করো। তেল খুব ভালো impedance match, আর disc-এর উপর কোনো শক্ত স্তর না থাকায় disc সবচেয়ে ভালো বাঁকে।
6. **Cable-এর মুখ seal:** cable যেখানে বের হচ্ছে সেখানে epoxy বা adhesive heat-shrink, যাতে পানি তারের ভেতর দিয়ে না ঢোকে।
7. **পরীক্ষা:** বালতির পানিতে রেখে বালতির গায়ে টোকা দাও, firmware-এর `scope`-এ level লাফালে ঠিক আছে। একদিন পানিতে রেখে দেখো multimeter-এ disc-এর দুই তারের মধ্যে resistance অসীম আছে কি না (leak নেই)।

> **Mounting:** hydrophone-কে ROV-এর frame থেকে নরম রাবার বা foam দিয়ে আলাদা করে লাগাও, thruster থেকে যত দূরে সম্ভব। Frame-এর মধ্য দিয়ে motor-এর কম্পন সরাসরি piezo-তে গেলে noise অনেক বাড়ে। নিজের speaker-এর box থেকেও দূরে রাখো।

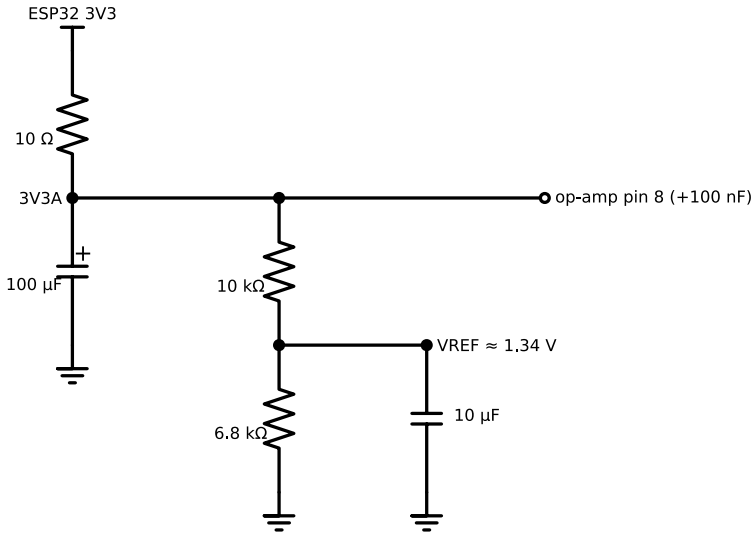
> **"Speaker-এর receiver"-এর ব্যাপারে:** একটা speaker-কেও উল্টো দিকে microphone হিসেবে ব্যবহার করা যায়, কিন্তু বন্ধ box-এর ভেতরে থাকলে পানির শব্দ খুব কম পৌঁছায় (বাতাসের ফাঁকের কারণে, 9.1)। Piezo সরাসরি পানির সংস্পর্শে থাকে, তাই অনেক ভালো। Receiver হিসেবে piezo-ই ব্যবহার করো।

10. Receiver preamp circuit



U1 = MCP6002 (3.3 V, best) or LM358 (power it from 5 V). Pin 8 = V+, pin 4 = GND, 100 nF across them.

চিত্র 10.1: দুই stage-এর non-inverting band-pass amplifier, তারপর anti-alias RC, তারপর ESP32 GPIO34।



চিত্র 10.2: Preamp-এর পরিষ্কার supply (3V3A) আর VREF (≈1.34 V)।

10.1 কেন এই circuit

- Piezo-র signal খুব ছোট (mV)। ESP32-এর ADC-তে ভালো করে মাপতে হলে কয়েকশো mV লাগে। তাই মোট gain 30–63 dB (৩০ থেকে ১৪০০ গুণ)।
- Single supply (শুধু 3.3 V) দিয়ে AC signal amplify করতে হলে signal-কে একটা মাঝামাঝি voltage (VREF)-এর উপর বসাতে হয়, যাতে signal উপরে-নিচে দুই দিকেই দুলতে পারে। Rb (1 MΩ) সেটা করে।
- Rg-Cg দিয়ে DC gain = 1 রাখা হয়েছে, তাই output-এর DC = VREF থাকে, gain বাড়ালেও DC সরে না।
- প্রতি stage নিজেই band-pass: নিচে Cg (pump hum কাটে), উপরে Cf (pinger ও ESC noise কাটে)।

- Ro-Caa: ADC-র আগে শেষ low-pass (anti-alias), আর op-amp-কে ADC-র input থেকে রক্ষা করে।

10.2 Formula ও মান

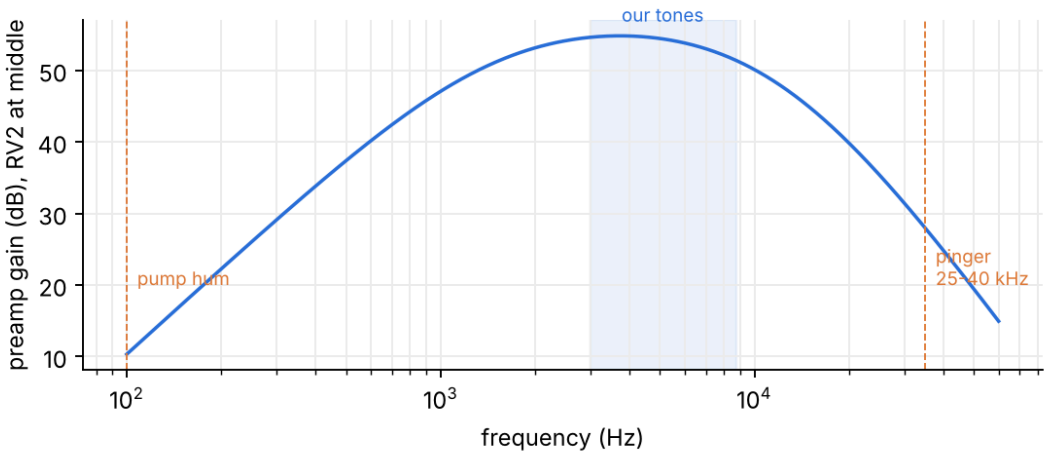
```
VREF = 3.3 × 6.8k / (10k + 6.8k) = 1.34 V
```

কেন 1.65 V (মাঝখান) না: ESP32 ADC (12 dB attenuation) প্রায় 0.15–2.45 V-এ সবচেয়ে সোজা (linear) মাপে। তার মাঝখান ≈ 1.3 V, দুই দিকে ±1.1 V জায়গা থাকে।

```
G = 1 + Rf / Rg (non-inverting gain)
Stage 1: G₁ = 1 + 100k/3.3k = 31.3 (29.9 dB)
Stage 2: G₂ = 1 + RV2/2.2k = 1 … 46.5 (0 … 33 dB, RV2 দিয়ে বদলানো যায়)
মোট: 30 dB … 63 dB
```

```
f_HP = 1/(2π Rg Cg): stage 1 → 1/(2π·3.3k·33n) = 1.46 kHz; stage 2 → 1/(2π·2.2k·47n) = 1.54 kHz
f_LP = 1/(2π Rf Cf): stage 1 → 1/(2π·100k·150p) = 10.6 kHz; stage 2 → 1/(2π·100k·100p) = 15.9 kHz (RV2 max)
Anti-alias: 1/(2π·3.3k·4.7n) = 10.3 kHz
Input HP: 1/(2π · Rb · (Cin ‖series C_piezo)) ≈ 10 Hz
```

ফলে 3–8.75 kHz-এর tone পুরো gain পায়। 100 Hz pump hum প্রায় 45 dB কমে, 35 kHz pinger প্রায় 27 dB কমে (চিত্র 10.3)।



চিত্র 10.3: Preamp-এর frequency response (RV2 মাঝামাঝি)। নীল অংশে আমাদের tone।

```
GBW ≥ G × f_max = 46.5 × 10 kHz ≈ 0.47 MHz
```

Op-amp-এর gain-bandwidth product। MCP6002 = 1 MHz ✔। LM358 ≈ 0.7–1 MHz, চলবে।

10.3 Op-amp বাছাই ও pin

	MCP6002 (ভালো)	LM358 (বিকল্প)
Supply	3V3A (3.3 V)	**5 V** (3.3 V-এ output 1.8 V-এর উপরে উঠতে পারে না)
Output swing	rail-to-rail (0–3.3 V)	0 থেকে Vcc−1.5 V
Pin	দুটোরই pin এক: 1 = OUT A, 2 = −IN A, 3 = +IN A, 4 = GND, 5 = +IN B, 6 = −IN B, 7 = OUT B, 8 = V+	

LM358 5 V-এ চালালে output 3.5 V পর্যন্ত যেতে পারে। Ro (3.3 kΩ) current সীমিত রাখে, তাই ESP32-এর pin নিরাপদ থাকে। তবুও MCP6002 পেলে সেটাই ব্যবহার করো। Pin 8 আর pin 4-এর মধ্যে যত কাছে সম্ভব 100 nF লাগাও।

> **GPIO34 কেন:** ESP32-এর DMA ADC শুধু ADC1-এ (GPIO32–39) কাজ করে। GPIO34 শুধু input pin, এতে কোনো internal pull-up নেই, তাই analog-এর জন্য একদম ঠিক।

11. Noise filtering: তিন স্তর

Noise-এর উৎস	কেমন	কোন স্তরে আটকায়
Pool pump/filter	নিচু frequency (<1–2 kHz), 100 Hz hum	Band choice + preamp HP
Thruster (প্রপেলার)	সব frequency জুড়ে, ঢেউয়ের মতো ওঠানামা	Narrowband Goertzel + whitening + code
ESC/motor whine	একটা নির্দিষ্ট tone (যেমন 8 kHz)	Per-tone noise floor (whitening)
টোকা, ধাক্কা (click)	খুব ছোট কিন্তু জোরালো	Interleaving + Viterbi
RoboSub pinger	25–40 kHz, 4 ms pulse	Preamp LP + Fs = 50 kHz (alias বাইরে পড়ে)
Echo (reverberation)	নিজের signal-এর দেরিতে আসা কপি	Frequency hopping + interleaving + code
বিদ্যুতের noise (buck, ESC)	তারে ঢোকা	Shielded cable, star ground, আলাদা buck, LC filter

11.1 স্তর 1: Hardware

Preamp band-pass (অধ্যায় 10), anti-alias RC, shielded cable (shield শুধু preamp-এর GND-এ, এক প্রান্তে), আলাদা power, hydrophone-কে frame থেকে কম্পন-বিচ্ছিন্ন রাখা।

11.2 স্তর 2: Sampling আর aliasing

> $Fs > 2 \cdot f_{max}$ (Nyquist)
>
> $f_{alias} = |f - k \cdot Fs|$ (k = সবচেয়ে কাছের পূর্ণসংখ্যা)
>
> ADC Fs-এর অর্ধেকের উপরের frequency ধরতে পারে না, সেগুলো ভুল করে নিচের frequency হিসেবে দেখা দেয় (alias)। Fs = 50 kHz হলে 35 kHz pinger → $|35 - 50|$ = 15 kHz, আমাদের band (3–8.75 kHz)-এর বাইরে ✔। Fs = 40 kHz হলে একই pinger → 5 kHz, ঠিক আমাদের tone-এর উপর ✘। তাই 50 kHz বেছে নেওয়া হয়েছে। Band-এ alias হয়ে ঢুকতে পারে শুধু 41.25–47 kHz-এর শব্দ, যা preamp ~31–33 dB কমিয়ে দেয়।

11.3 স্তর 3: Software (DSP)

- **Goertzel (narrowband filter):** প্রতিটা tone-এর জন্য মাত্র ~94 Hz চওড়া একটা "জানালা" দিয়ে শোনে। 6 kHz band-এর noise-এর মাত্র 94/6000 ভাগ ঢোকে। এটাই processing gain:

> $PG = 10 \log_{10}(B_{band} / B_{bin}) = 10 \log_{10}(6000 / 94) \approx 18$ dB
>
> তাই wideband SNR −6 dB হলেও প্রতি tone-এ ~১০ dB SNR পাওয়া যায় (Hann window আর echo-র ক্ষতি বাদ দিয়ে)।

- **Whitening (per-tone noise floor):** প্রতিটা tone-এর জন্য আলাদা noise floor মনে রাখা হয়। কোনো tone-এর জায়গায় সবসময় একটা whine থাকলে সেই tone-এর floor বেড়ে যায়, তাই whine আর "জয়ী" হয় না (অধ্যায় 14.3)।
- **Ratio-ভিত্তিক সিদ্ধান্ত:** কোন tone বেজেছে, সেটা নির্ধারণ হয় চারটা tone-এর মধ্যে তুলনা করে, কোনো fixed threshold দিয়ে না। তাই signal জোরে বা আস্তে, যাই আসুক, কাজ করে।

11.4 স্তর 4: Protocol

- **Frequency hopping:** পরপর slot আলাদা frequency set ব্যবহার করে। ফলে আগের slot-এর echo পরের slot-এর হিসাবে ঢোকে না।
- **Interleaving + convolutional code:** একটা slot নষ্ট হলে বা একটা frequency fade করলে মাত্র কয়েকটা ছড়ানো bit ভুল হয়, Viterbi সেগুলো ঠিক করে ফেলে।
- **CRC-16:** ভুল packet প্রায় কখনো পাস করে না (৬৫৫৩৬-এ ১)।
- **ACK + retry:** উত্তর না এলে firmware নিজেই আবার পাঠায় (৩ বার পর্যন্ত)।

12. Signal design: tone, slot, hopping

12.1 Tone-এর তালিকা

```
f_k = 3000 + 250 · k Hz, k = 0 … 23 → 3000, 3250, …, 8750 Hz
```

12.2 কেন 250 Hz পরপর, কেন 16 ms window

```
N = Fs × T_win = 50000 × 0.016 = 800 sample
Δf_bin = Fs / N = 50000 / 800 = 62.5 Hz
250 Hz = 4 × 62.5 Hz → প্রতিটা tone ঠিক একটা "bin"-এর উপর, পাশাপাশি tone-এর মধ্যে 4 bin ফাঁক
```

একই window-এ পূর্ণসংখ্যক bin দূরের দুটো tone একে অপরকে "দেখে" না (orthogonal)। 4 bin ফাঁক থাকায় ADC-র clock একটু ভুল হলেও (±1–2%) বা Hann window দিলেও tone মিশে যায় না।

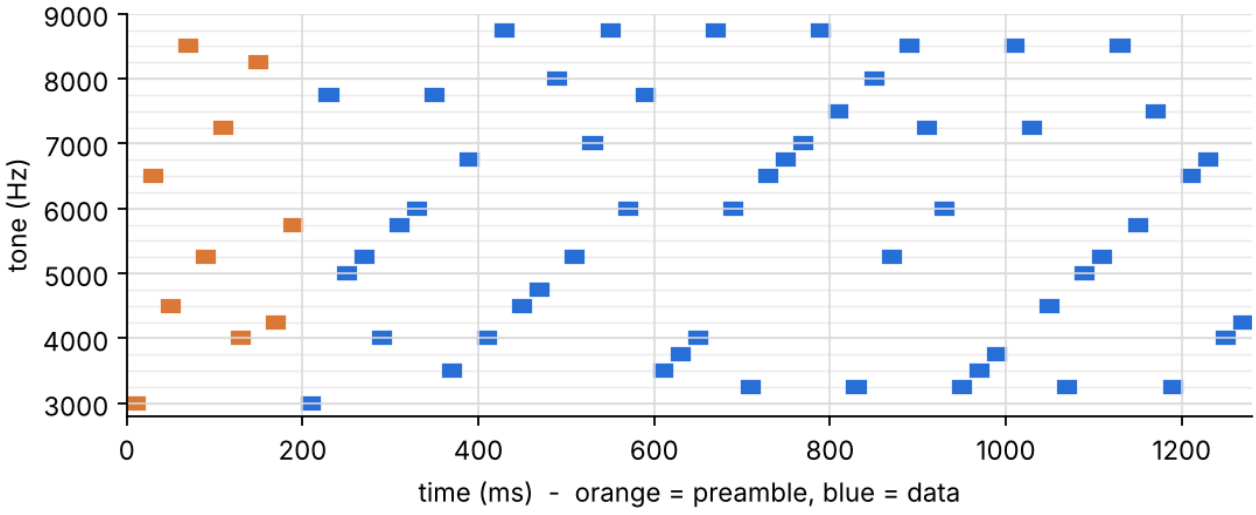
12.3 Slot আর window

প্রতিটা slot 20 ms, কিন্তু receiver মাঝের 16 ms (800 sample) দেখে। দুই পাশে 2 ms করে margin থাকে, যাতে timing একটু এদিক-ওদিক হলেও পাশের slot ঢুকে না পড়ে।

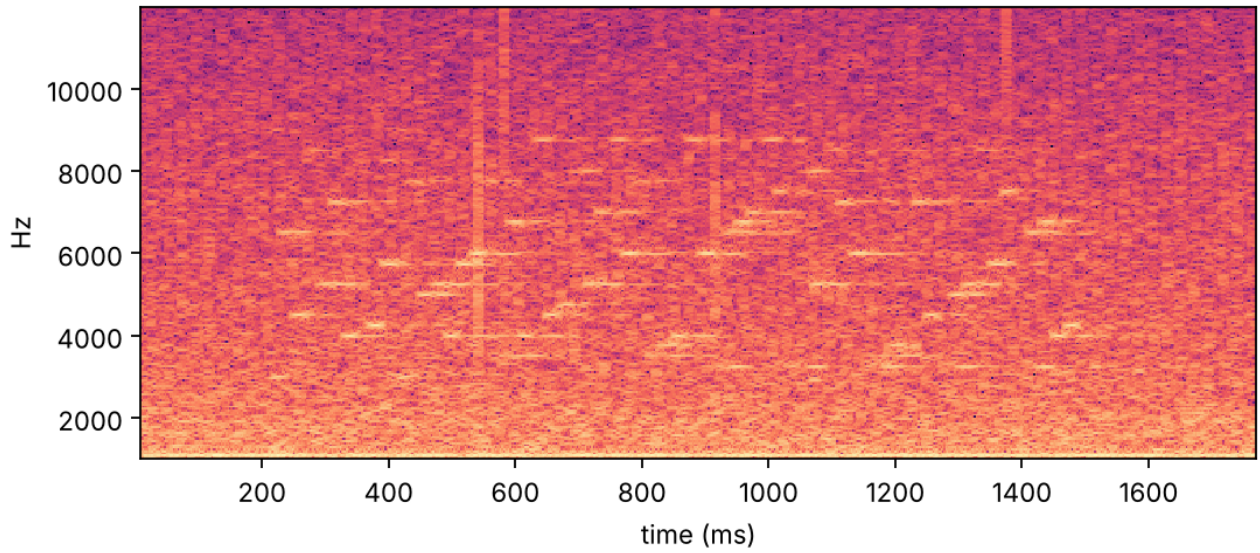
12.4 Frequency hopping set

```
set s = { s, s+6, s+12, s+18 }, s = 0 … 5
slot i ব্যবহার করে set (i mod 6); 2 bit মান v (0–3) → tone = s + 6v
```

উদাহরণ: set 0 = {3000, 4500, 6000, 7500 Hz}, set 1 = {3250, 4750, 6250, 7750 Hz}। একই set আবার আসে 6 slot = 120 ms পরে, ততক্ষণে আগের echo অনেক কমে যায় (চিত্র 2.1)। Set-এর ভেতরে tone-গুলো 1500 Hz দূরে, তাই কোনো এক জায়গার fading একসাথে চারটাকে নষ্ট করে না।



চিত্র 12.1: একটা packet-এর সময়-frequency নকশা। কমলা = preamble (10 slot), নীল = data (54 slot)।



চিত্র 12.2: Virtual pool-এ (echo + pump + thruster + pinger, SNR 0 dB) ADC যা দেখে তার spectrogram।

12.5 Preamble

Preamble হলো 10টা আলাদা tone-এর একটা নির্দিষ্ট ক্রম: 3000, 6500, 4500, 8500, 5250, 7250, 4000, 8250, 4250, 5750 Hz। Receiver সারাক্ষণ এই ক্রমটা খোঁজে। এটা বেছে নেওয়া হয়েছে তিনটা নিয়ম মেনে (code-এ test আছে):

1. পাশাপাশি slot-এর tone অনেক দূরে, যাতে একটার echo পরেরটার মতো না শোনায়।
2. অন্য packet-এর data কোনোভাবেই 10টার মধ্যে 2টার বেশি slot মেলাতে পারে না, তাই data দেখে ভুল করে "packet শুরু" ধরে নেওয়া হয় না।
3. Preamble-এর কোনো tone-এর set, সেই packet-এর data অন্তত 6 slot (120 ms) পরে ব্যবহার করে। Simulation-এ প্রথম version-এ এই নিয়ম ছিল না, তখন preamble-এর echo প্রথম data slot নষ্ট করছিল। এটা ধরা পড়ার পর ঠিক করা হয়েছে।

12.6 Data rate

> `raw = 2 bit / 20 ms = 100 bit/s; code rate ½ → 50 bit/s`
>
> একটা `packet = 64 slot × 20 ms = 1.28 s`, এতে `4 byte payload (32 bit) → কার্যকর ≈ 25 bit/s`
>
> কম মনে হতে পারে, কিন্তু "LED on", "start mission 3", "আমি gate পার হয়েছি" জাতীয় command-এর জন্য যথেষ্ট, আর echo-ভর্তি pool-এ নির্ভরযোগ্যতা অনেক বেশি জরুরি।

13. Packet, CRC, error correction

13.1 Packet-এর গঠন (6 byte)

Byte	বিট 7–4	বিট 3–0
0	dst (কাকে, 15 = সবাইকে)	src (কে পাঠাল)
1	seq (0–15, repeat চেনার জন্য)	flags: 8 = ACK চাই, 4 = এটা ACK
2	cmd (0–255): 1 = LED on, 2 = LED off, 3 = toggle, 4 = ping, 10–255 তোমার নিজের	
3	arg (0–255)	
4–5	CRC-16 (byte 0–3-এর উপর)	

13.2 CRC-16

> $g(x) = x^{16} + x^{12} + x^5 + 1$ (0x1021), শুরুর মান 0xFFFF ("CRC-16/CCITT-FALSE")
>
> Data-কে একটা বড় binary সংখ্যা ধরে g(x) দিয়ে ভাগ করলে যে ভাগশেষ থাকে, সেটাই CRC। Receiver নিজে আবার হিসাব করে মেলায়। এক-দুই bit ভুল সবসময় ধরা পড়ে, এলোমেলো ভুল ধরা পড়ে 65535/65536 বার। পরীক্ষার মান: "123456789" → 0x29B1 (unit test-এ যাচাই করা)।

13.3 Convolutional code (K = 7, rate ½)

48 bit (6 byte)-এর পরে 6টা শূন্য ("tail") যোগ করে 54 bit বানানো হয়। প্রতিটা bit একটা 7-ঘরের shift register-এ ঢোকে, আর প্রতি bit-এ 2টা output bit বের হয়:

> reg = [নতুন bit, আগের 6 bit]
>
> out_0 = parity(reg AND 1111001_2) (171 octal)
>
> out_1 = parity(reg AND 1011011_2) (133 octal)
>
> parity = কয়টা 1 আছে তা বিজোড় হলে 1। প্রতিটা output bit আগের 7টা input bit-এর উপর নির্ভর করে, তাই তথ্য অনেকগুলো coded bit-এ ছড়িয়ে থাকে। এই code-এর "free distance" 10: দুটো আলাদা message-এর coded রূপ কমপক্ষে 10টা bit-এ আলাদা। তাই কাছাকাছি নয় এমন অনেক ভুল ঠিক করা যায়। এটা NASA আর Wi-Fi-তে ব্যবহৃত বিখ্যাত code। প্রথমে Hamming(8,4) code দিয়ে design করা হয়েছিল। Simulation-এ দেখা গেল convolutional code একই noise-এ অনেক বেশি packet ঠিক আনে (−6 dB-এ ২৫% থেকে ৯০%+), তাই এটা বেছে নেওয়া হয়েছে।

13.4 Interleaver

> slot(c) = (37 · c) mod 54; c < 54 হলে MSB, নাহলে LSB
>
> c = coded bit-এর নম্বর (0–107)। পাশাপাশি coded bit (যেগুলো Viterbi-র একসাথে লাগে) অন্তত 3 slot দূরে আর আলাদা frequency set-এ যায়। ফলে একটা ঝাঁকুনি (click) বা একটা faded frequency শুধু ছড়ানো কয়েকটা bit নষ্ট করে, যা Viterbi সবচেয়ে ভালো ঠিক করতে পারে।

14. Receiver কীভাবে decode করে (ধাপে ধাপে)

14.1 DC সরানো

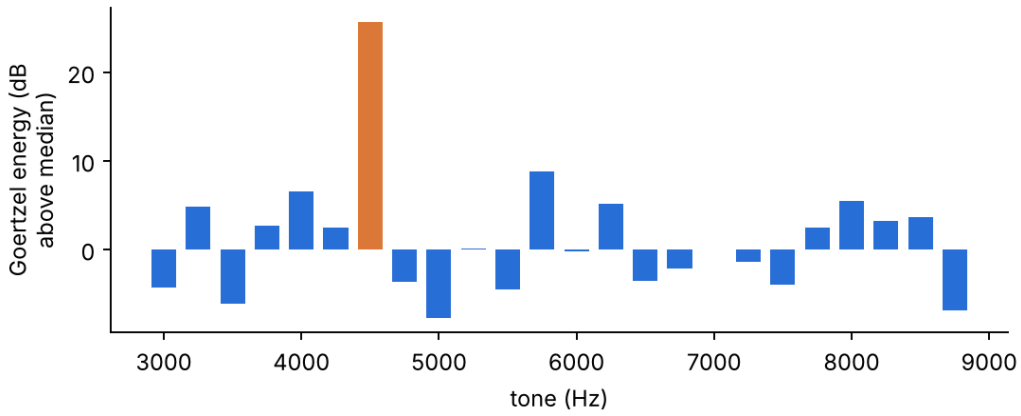
```
dc ← dc + (x − dc) / 1024;  y = x − dc
f_c ≈ Fs / (2π · 1024) = 50000 / 6434 ≈ 7.8 Hz
```

ADC-র sample-এ VREF (≈1600 count)-এর DC থাকে। এই এক লাইনের high-pass filter সেটা সরায়।

14.2 Goertzel: একটা tone-এর শক্তি মাপা

```
w[n] = 0.5 − 0.5 cos(2π (n + 0.5) / N)   (Hann window)
coef = 2 cos(2π f / Fs)
s[n] = x[n]·w[n] + coef · s[n−1] − s[n−2]
E = s[N−1]² + s[N−2]² − coef · s[N−1] · s[N−2]
```

Goertzel হলো FFT-এর একটা মাত্র bin হিসাব করার সস্তা উপায়: প্রতি sample-এ একটা গুণ আর দুটো যোগ। আমাদের শুধু 24টা frequency দরকার, পুরো FFT লাগে না। Hann window কিনারাগুলো নরম করে দেয়, তাই পাশের tone-এর "leakage" কমে (Fs একটু ভুল হলেও)।



চিত্র 14.1: 4500 Hz tone + noise-এর উপর 24টা Goertzel। কমলা bin স্পষ্ট জয়ী।

14.3 Whitening: প্রতিটা tone-এর নিজস্ব noise floor

```
N_k ← N_k + (min(E_k, 8·N_k) − N_k) / 256   (প্রতি 2.5 ms)
z_k = E_k / N_k
SNR_k ≈ z_k − 1,   q = 10 log₁₀(z − 1) dB
```

N_k হলো tone k-এর গড় noise (exponential moving average, সময় ধ্রুবক 256 × 2.5 ms = 0.64 s)। শুধু packet না থাকার সময় শেখে। "min(..., 8N)" মানে হঠাৎ জোরালো শব্দ (click বা নিজের signal) floor-কে বেশি টেনে তুলতে পারে না। z_k বলে tone k তার স্বাভাবিক noise-এর কত গুণ। ধ্রুব whine থাকলে N_k বড় হয়ে যায়, ফলে whine-এর z ≈ 1 থাকে। Firmware যে **q** print করে, সেটা এই SNR-এর median।

14.4 Preamble খোঁজা

প্রতি 2.5 ms (H = 125 sample) পরপর শেষ 16 ms-এর 24টা z হিসাব হয়। প্রতিটা preamble slot j-এর tone p_j কতটা "জয়ী", তা ভাগ হিসেবে মাপা হয়:

```
frac_j(t) = z_{p_j}(t) / Σ_k z_k(t)
score(t) = (1/10) Σ_{j=0..9} frac_j( t − (9 − j)·T_slot )
match(t) = কয়টা j-তে p_j-ই 24টার মধ্যে সবচেয়ে জোরালো
শর্ত: score ≥ 0.14 এবং match ≥ 5
```

শুধু noise থাকলে প্রতি frac ≈ 1/24 ≈ 0.04, তাই score ≈ 0.04। Preamble থাকলে score 0.3–0.9। ভাগ ব্যবহার করায় পুরো band-জুড়ে হঠাৎ শব্দ (click) এলে সব z একসাথে বাড়ে, score বাড়ে না। দুই শর্ত একসাথে থাকায় 10 মিনিটের কঠিন noise-এ একবারও ভুল trigger হয়নি (অধ্যায় 17)।

14.5 Timing ঠিক করা

Score-এর চূড়া একটু চ্যাপ্টা (window slot-এর চেয়ে ছোট বলে)। Trigger-এর পরে আরো 14টা hop (35 ms)-এর score রাখা হয়। তারপর চূড়ার 85%-এর উপরের অংশের ভারকেন্দ্র (centroid) নেওয়া হয়:

```
c = Σ i · (score_i − 0.7·max) / Σ (score_i − 0.7·max)
data slot i-এর window শেষ হয়: e_i = sync + (i + 1) · S, S = Fs × 0.020
```

Data চলার সময় **early/late gate** timing-কে আস্তে আস্তে ঠিক রাখে। জয়ী tone-এর শক্তি 1 ms আগের ও পরের window-এ মাপা হয়, আর যেদিকে বেশি সেদিকে একটু সরানো হয়:

```
err = (E_late − E_early) / (E_late + E_early); adj ← adj + 0.25 · D · err (D = 1 ms, সীমা ±5 ms)
```

14.6 ADC-র আসল sample rate মাপা

```
Fs_measured = (মোট sample সংখ্যা) / (সময়, µs) × 10⁶
timing ভুল = (Fs ভুল) × packet দৈর্ঘ্য
```

ESP32-এর ADC ঠিক 50000 না, একটু কম-বেশি দেয়। 1% ভুল হলে 1.28 s packet-এর শেষে timing 12.8 ms সরে যায়, যা slot-এর margin (2 ms)-এর অনেক বেশি। তাই firmware চলার সময় ক্রমাগত sample গোনে আর ESP32-এর নিজের µs timer-এর সাথে তুলনা করে। সময় যত বাড়ে, মাপ তত নিখুঁত হয় (20 s পরে ≈0.05% → packet-এর শেষে 0.6 ms)। Simulation-এ ADC-কে ইচ্ছা করে 2.4% দ্রুত চালিয়ে test করা হয়েছে।

14.7 Soft bit (LLR)

```
L_MSB = max(ln z₀, ln z₁) − max(ln z₂, ln z₃)
L_LSB = max(ln z₀, ln z₂) − max(ln z₁, ln z₃)
```

L > 0 মানে bit সম্ভবত 0, L < 0 মানে 1, আর |L| যত বড় তত নিশ্চিত। শুধু 0/1 না দিয়ে কতটা নিশ্চিত সেটাও Viterbi-কে দেওয়া হয় ("soft decision")। এতে hard decision-এর চেয়ে প্রায় 2 dB বেশি noise সহ্য হয়।

14.8 Viterbi decoding

```
PM_new(s′) = max over predecessor s of [ PM(s) + (±L₀) + (±L₁) ]
```

64টা সম্ভাব্য register অবস্থার প্রতিটার জন্য এখন পর্যন্ত সবচেয়ে মানানসই পথটা মনে রাখা হয় (PM = path metric)। প্রত্যাশিত bit 0 হলে +L, 1 হলে −L যোগ হয়। শেষে tail-এর কারণে অবস্থা 0-তে শেষ হওয়া পথটা পিছন দিকে (traceback) পড়ে 54 bit ফিরে পাওয়া যায়। এটাই maximum-likelihood decoding: যে message সবচেয়ে সম্ভাব্য, সেটাই বের হয়।

14.9 CRC আর কাজ

প্রথম 48 bit থেকে 6 byte বানিয়ে CRC মেলানো হয়। মিললে packet queue-তে যায়, main loop LED জ্বালায় বা নিভায়, দরকার হলে ACK পাঠায়। আর একই (src, seq) 15 s-এর মধ্যে আবার এলে command দ্বিতীয়বার চালানো হয় না (repeat), কিন্তু ACK আবার পাঠানো হয়, কারণ আগের ACK হয়তো হারিয়ে গিয়েছিল।

15. Firmware: install, upload, ব্যবহার

15.1 Install

1. Arduino IDE 2.x install করো।
2. File → Preferences → "Additional boards manager URLs"-এ বসাও: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Tools → Board → Boards Manager → "esp32 by Espressif Systems" **version 3.x** install করো (3.3.12-এ compile করে দেখা হয়েছে; 2.x চলবে না, code নিজেই error দেবে)।
4. Repository-র `firmware/DuburiModem` folder খোলো (`DuburiModem.ino`)। একই folder-এ `am_core.c`, `am_core.h`, `config.h` থাকতে হবে।
5. Tools → Board → "ESP32 Dev Module"। কোনো বাড়তি library লাগে না।
6. `config.h`-এ `NODE_ID` বসাও: প্রথম ROV-এ 1, দ্বিতীয়টায় 2। আর `PEER_ID`-এ অন্যটার ID।
7. Upload। Serial Monitor: 115200 baud, line ending "Newline"।

15.2 Serial command

Command	কাজ
`on [id]` / `off [id]` / `toggle [id]`	id-এর ROV-এর LED জ্বালাও/নিভাও/উল্টাও (id না দিলে PEER_ID, 15 = সবাই)
`ping [id]`	উত্তর চায়; উত্তরের q দিয়ে link কেমন বোঝা যায়
`send C A [id]`	নিজের command C (0–255) আর argument A পাঠাও
`sweep`	24টা tone একটা একটা করে 0.3 s বাজায় (frequency response test)
`tone F MS`	F Hz, MS millisecond বাজাও
`scope`	প্রতি 0.25 s-এ 24টা tone-এর whitened level (dB) দেখায় (আবার লিখলে বন্ধ)
`stats`	কয়টা preamble, কয়টা ঠিক, কয়টা CRC fail, ADC-র মাপা Fs, level, clipping
BOOT button	PEER_ID-কে "toggle" পাঠায় (laptop ছাড়াই test)

15.3 Output-এর format (main computer-এর জন্য)

```
TX to=2 seq=3 cmd=1 arg=0 (try 1/3)
RX src=1 dst=2 seq=3 cmd=1 arg=0 flags=8 q=14.2dB      ← receiver-এ
TX ack to=1 seq=3
ACK src=2 seq=3 cmd=1 state=1 q=12.8dB                  ← sender-এ
OK delivered to 2 (try 1)
```

Jetson বা Raspberry Pi একই USB serial port থেকে এগুলো পড়তে আর command লিখতে পারে। Python উদাহরণ:

```
import serial                                  # pip install pyserial
s = serial.Serial("/dev/ttyUSB0", 115200, timeout=0.1)
s.write(b"send 10 3 2\n")                       # ROV 2-কে command 10, argument 3
while True:
    line = s.readline().decode(errors="ignore").strip()
    if line.startswith("RX "):
```

```
        f = dict(kv.split("=") for kv in line.split()[1:])
        print("command", f["cmd"], "from", f["src"], "quality", f["q"])
```

15.4 নিজের command যোগ করা

`DuburiModem.ino`-তে `enum { CMD_LED_ON = 1, … }`-এ নতুন নাম দাও (10–255), তারপর `handlePacket()`-এর `switch (p.cmd)`-এ `case` লেখো। পাঠানোর জন্য `sendCommand(dst, CMD_X, arg)` বা serial-এ `send`।

16. Code-এর ব্যাখ্যা (function ধরে ধরে)

16.1 File-এর ভাগ

File	কী আছে
`config.h`	শুধু এটাই সাধারণত বদলাতে হয়: ID, pin, ACK, retry, heat guard
`am_core.h / am_core.c`	Modem-এর মস্তিষ্ক, সাধারণ C, ESP32-এর কিছু নেই। একই file PC test-এ চলে।
`DuburiModem.ino`	ESP32-এর অংশ: ADC DMA, LEDC tone, FreeRTOS task, serial command, LED, ACK
`tests/test_core.c`	PC-তে unit test (CRC, code, interleaver, loopback, noise)
`sim/pool_sim.py, run_sim.py`	Virtual pool, আসল C code চালিয়ে ফল আর graph
`docs/`	এই PDF-এর উৎস, চিত্র, font

16.2 am_core.c

- `am_tone_hz(k)` : f = 3000 + 250k।
- `am_crc16()` : bit ধরে ধরে CRC হিসাব (13.2)।
- `am_conv_encode()` : 7-ঘরের register, দুটো parity (13.3)।
- `am_viterbi()` : 64 state, প্রতি step-এ প্রতি state-এর দুটো আগের state থেকে ভালোটা বাছা, শেষে traceback (14.8)। `dec[t]`-এর প্রতিটা bit বলে কোন আগের state বাছা হয়েছিল।
- `am_interleave(c)` : slot = 37c mod 54 (13.4)।
- `am_pack()/am_unpack()` : packet ↔ 6 byte, CRC বসানো/মেলানো।
- `am_tx_build()` : packet → bit → code → interleave → 64টা tone নম্বর (প্রথম 10টা preamble)।
- `rx_configure()` : Fs থেকে W (800), H (125), D (50), S (1000), Hann window, Goertzel coef, preamble offset।
- `am_rx_process()` : প্রতি sample-এ clipping গোনা, DC সরানো, ring buffer-এ রাখা। পরের "event"-এর সময় হলে `rx_hop()` (খোঁজা) বা `rx_decode_slot()` (data) ডাকা।
- `rx_energies()` : ring buffer থেকে W sample নিয়ে window গুণ করে, যতগুলো tone দরকার তাদের Goertzel।
- `rx_hop()` : 24 tone-এর E, noise floor update, z, frac, score, match। Trigger হলে PEAK অবস্থা।
- `rx_finish_peak()` : চূড়ার centroid → sync সময়, DECODE অবস্থা শুরু।
- `rx_decode_slot()` : slot-এর set-এর 4টা tone, z, LLR, quality, early/late।
- `rx_finish_packet()` : de-interleave, Viterbi, byte, CRC, quality median, queue-তে রাখা, আবার খোঁজা শুরু।
- `am_rx_mute()` : নিজে পাঠানোর সময় শোনা বন্ধ (নিজের জোরালো শব্দ যেন ভুল কিছু না ঘটায়)।
- `am_rx_set_fs()` : মাপা Fs বসানো, শুধু খোঁজার অবস্থায় (packet চলার মাঝে না)।

16.3 DuburiModem.ino

- `startAdc()`: ESP-IDF-এর `adc_continuous` driver। GPIO34 → ADC1 channel, 12 dB attenuation, 50 kHz, DMA frame 512 sample, overflow হলে গোনা হয়।
- `rxTask()` (core 0, priority 5): DMA থেকে sample পড়া, level মাপা, mutex নিয়ে `am_rx_process()`, packet হলে queue-তে পাঠানো। প্রতি 2 s-এ Fs মাপা আর দরকার হলে বসানো।
- `playTones()`: heat guard, RX mute, amp চালু, তারপর প্রতি slot-এ `ledcWriteTone()`। সময় মাপা হয় `esp_timer` (µs) দিয়ে, প্রতি slot-এর শেষ "absolute" সময়ে, তাই drift জমে না।
- `heatGuard()/heatGuardUpdate()`: 7.2-এর formula।
- `sendCommand()`: seq বাড়ানো, পাঠানো, ACK-এর জন্য 2.6 s অপেক্ষা, না এলে আবার (মোট 3 বার)।
- `handlePacket()`: আমার জন্য কি না দেখা, ACK চেনা, repeat চেনা, command চালানো, ACK পাঠানো।
- `loop()`: serial line পড়া, BOOT button, scope print, packet poll।

> **Thread safety:** modem-এর state (`g_rx`) দুই task ব্যবহার করে, তাই প্রতিবার mutex (`g_rx_lock`) নেওয়া হয়। Packet main loop-এ যায় FreeRTOS queue দিয়ে। Heavy DSP core 0-তে চলে, serial আর TX core 1-এ।

17. Verification: কী test করা হয়েছে

17.1 Compile

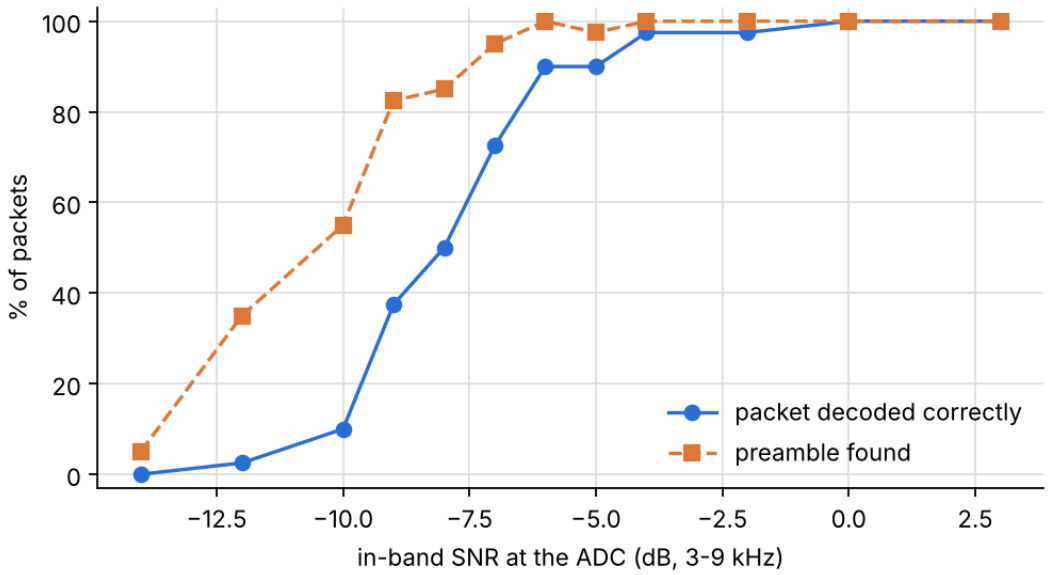
Firmware Arduino-ESP32 core 3.3.12 (ESP-IDF 5.5) দিয়ে "ESP32 Dev Module"-এর জন্য compile করা হয়েছে, সব warning চালু রেখে। কোনো warning বা error নেই। Flash-এর ~23% আর RAM-এর ~17% লাগে।

17.2 Unit test (PC-তে, `make -C tests && tests/test_core`)

- CRC-16-এর মান "123456789" → 0x29B1।
- Convolutional code + Viterbi: 3000 বার এলোমেলো data, 0–4টা ছড়ানো ভুল, সবই ঠিক হয়েছে। এক bit বদলালে ≥10 coded bit বদলায়।
- Interleaver একটা permutation, পাশাপাশি bit আলাদা slot আর set-এ।
- Pack/unpack 2000 packet, প্রতিটায় 1 bit উল্টালে CRC ধরে।
- Preamble-এর তিনটা নিয়ম (12.5)।
- Loopback: পরিষ্কার signal, square wave, 1-sample-করে input, ADC 1.5% দ্রুত, Fs মাপায় 0.05% ভুল, 48 kHz, প্রায় 0 dB noise।
- 60 s শুধু noise: একটাও ভুল packet নেই।

17.3 Virtual pool simulation

ROV-এর আসল C code-কে একটা কম্পিউটার-বানানো pool-এর signal খাওয়ানো হয়েছে। Model-এ ছিল: ESP32 LEDC-এর আসল frequency ধাপ, slot-এর সময় ±30 µs কাঁপা, RC filter, speaker-এর অজানা response (1.5–7 kHz band, এলোমেলো ±5 dB ঢেউ), 50×25×5 m pool-এ image-method echo (দেয়াল/মেঝে reflection 0.85, পানির উপরিতল −0.9), 10–25 m এলোমেলো অবস্থান, pump hum, thruster noise (ওঠানামা সহ), click, 30 kHz pinger, preamp-এর filter, ADC (2.4% দ্রুত clock, 12 bit, noise)। প্রতি বিন্দুতে 40টা packet।



চিত্র 17.1: SNR অনুযায়ী কত packet ঠিক এসেছে। SNR = ADC-তে 3–9 kHz band-এর signal power / noise power।

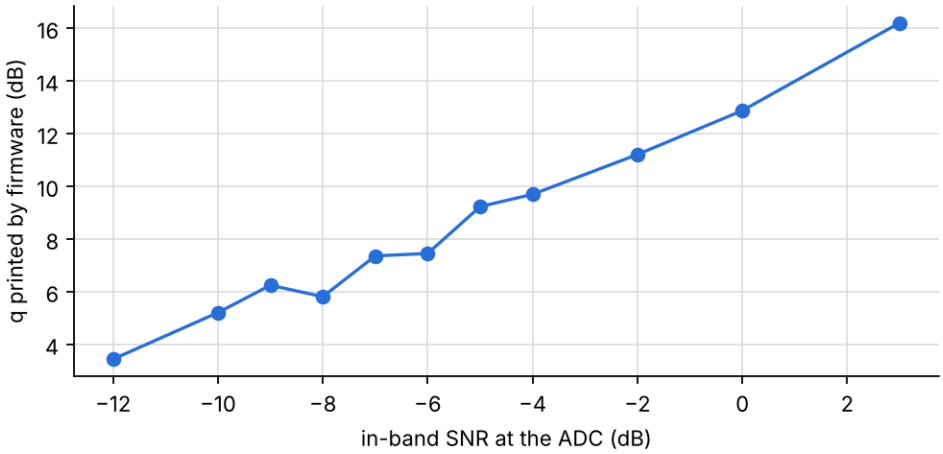
In-band SNR	Preamble পাওয়া গেছে	Packet একদম ঠিক	Firmware-এর q (median)
-14 dB	2/40	**0/40** (0%)	—
-12 dB	14/40	**1/40** (2%)	3.4 dB
-10 dB	22/40	**4/40** (10%)	5.2 dB
-9 dB	33/40	**15/40** (38%)	6.2 dB
-8 dB	34/40	**20/40** (50%)	5.8 dB
-7 dB	38/40	**29/40** (72%)	7.4 dB
-6 dB	40/40	**36/40** (90%)	7.5 dB
-5 dB	39/40	**36/40** (90%)	9.2 dB
-4 dB	40/40	**39/40** (98%)	9.7 dB
-2 dB	40/40	**39/40** (98%)	11.2 dB
+0 dB	40/40	**40/40** (100%)	12.9 dB
+3 dB	40/40	**40/40** (100%)	16.2 dB

Echo-র মাত্রা (SNR −3 dB)	ঠিক
দেয়ালের reflection = 0.5	**40/40**
দেয়ালের reflection = 0.7	**40/40**
দেয়ালের reflection = 0.85	**39/40**
দেয়ালের reflection = 0.93	**34/40**

Interference (SNR −3 dB)	ঠিক
baseline	**40/40**
ESC whine 5500 Hz	**40/40**
pinger 35 kHz	**40/40**
no thrusters	**39/40**

ADC clock 2.4% দ্রুত, SNR 0 dB	ঠিক
Fs মাপার ভুল 0.0% , timing tracker চালু	**20/20**
Fs মাপার ভুল 0.1% , timing tracker চালু	**20/20**
Fs মাপার ভুল 0.3% , timing tracker চালু	**20/20**
Fs মাপার ভুল 1.0% , timing tracker চালু	**16/20**
Fs মাপার ভুল 0.3% , timing tracker বন্ধ	**20/20**
Fs মাপার ভুল 0.1% , timing tracker বন্ধ	**20/20**

> **False alarm:** 600 s শুধু noise (pump + thruster + 5.5 kHz whine + click + 35 kHz pinger): preamble trigger 0, ভুল packet 0।



চিত্র 17.2: Firmware যে q print করে বনাম আসল SNR। মাঠে q দেখে বোঝা যায় link কতটা নিরাপদ।

> **মাঠের নিয়ম:** q ≥ 10 dB → নিশ্চিন্ত। q 7–10 dB → চলবে, কিছু retry হবে। q < 7 dB → সীমানায়, volume/gain/অবস্থান ঠিক করো। Fs-এর মাপ 1% ভুল হলে (16/20) ক্ষতি হয়, তাই firmware-এর ক্রমাগত Fs মাপা জরুরি। চালুর পর প্রথম ~10 s মাপ স্থির হতে দাও।

17.4 Simulation-এ পাওয়া আর ঠিক করা সমস্যা

1. Preamble-এর শেষ tone-এর echo প্রথম data slot নষ্ট করছিল। নিয়ম 3 (12.5) যোগ করে ঠিক করা হয়েছে।
2. প্রথম interleaver একটা codeword-এর সব bit একই frequency set-এ পাঠাচ্ছিল। এক frequency fade করলে পুরো codeword নষ্ট হতো। নতুন interleaver দিয়ে ঠিক করা হয়েছে।
3. Hamming(8,4)-এর বদলে convolutional code: −6 dB-এ সফলতা প্রায় ২৫% থেকে ৯০%।
4. Detection threshold 0.17 থেকে 0.14: −9 dB-এ preamble পাওয়া 75% থেকে 100%, false alarm-ও শূন্য।

17.5 কী test করা হয়নি (সৎভাবে)

- আসল ESP32, আসল speaker, আসল পানি। Code compile হয়েছে আর logic simulation-এ test হয়েছে, কিন্তু board-এ চালানো হয়নি। অধ্যায় 18-এর ধাপ ধরে নিজেরা test করবে।
- Car speaker-এর box থেকে পানিতে আসলে কত জোরে শব্দ যায়, আর pool-এর আসল noise কত। Simulation-এ SNR-কে চলক রাখা হয়েছে। মাঠে `scope` আর `q` দিয়ে মাপবে।
- Component-এর মান datasheet-এর সাধারণ মান থেকে, আর thermal ও plate হিসাব আনুমানিক।

এর বাইরে, আলাদা AI reviewer agent দিয়ে code, circuit আর এই document স্বাধীনভাবে যাচাই করানো হয়েছে। ফলাফল repository-র `docs/VERIFICATION.md`-এ আছে।

18. বানানো ও test করার ধাপ

ধাপ 1: Bench (পানি ছাড়া)

1. দুটো ESP32-এ firmware দাও (NODE_ID 1 আর 2)। Serial-এ "ready" দেখাবে।
2. শুধু preamp: piezo লাগিয়ে `stats` দেখো। LEVEL-এর mean ≈ 1500–1800, rms ছোট (5–30)। Piezo-তে আঙুলের টোকা দিলে max লাফাবে।
3. **Wire loopback:** node 1-এর RV1-এর wiper থেকে signal নিয়ে 100 kΩ আর 1 kΩ-এর divider দিয়ে ১০০ ভাগের ১ ভাগ করো (wiper → 100 kΩ → বিন্দু → 1 kΩ → GND)। সেই বিন্দু থেকে node 2-এর preamp input-এ (piezo-র জায়গায়) দাও। RV2 কম রাখো। দুই board-এর GND জোড়া দাও, আর speaker খুলে রাখো। Node 1-এ `on` লিখলে node 2-এর LED জ্বলবে। (ACK-এর জন্য উল্টো দিকেও একই তার লাগবে। না দিলে "no reply" দেখাবে, কিন্তু LED জ্বলবে।) এতে পুরো DSP chain পানি ছাড়াই পরীক্ষা হয়।
4. Amp + speaker বাতাসে, RV1 একদম কম। `tone 5000 500` দিয়ে শব্দ শোনো। আস্তে বাড়াও, কর্কশ হওয়ার আগে থামো।

ধাপ 2: বালতি

1. Speaker box আর hydrophone বালতিতে, 30–50 cm দূরে। Volume কম।
2. RX-এ `scope` চালু, TX-এ `sweep`। কোন tone-এ level কম আসে দেখো।
3. `stats`-এ clipped বাড়লে RV2 (gain) কমাও। rms খুব কম হলে বাড়াও।
4. `ping` ১০ বার। সবগুলো OK আর q বড় (>20 dB) হওয়া উচিত।

ধাপ 3: Pool

1. 5, 10, 15, 20, 25 m-এ প্রতিবার ২০টা `ping`। কয়টা OK আর q-এর মান লিখে রাখো।
2. Thruster বন্ধ আর চালু, দুইভাবেই। চালু অবস্থায় RX-এর `scope`-এ কোনো tone-এ স্থায়ী উঁচু level থাকলে বুঝবে সেখানে whine আছে (whitening সামলাবে, কিন্তু জেনে রাখা ভালো)।
3. প্রতি ১০ মিনিট পর speaker-এর resistance মেপে তাপমাত্রা দেখো (7.6)।
4. q < 7 dB হলে: (ক) volume একটু বাড়াও (clip না করে); (খ) hydrophone-কে thruster থেকে দূরে নাও; (গ) speaker জানালা পাতলা বা বড় করো; (ঘ) দরকার হলে PAM8610।

ধাপ 4: ROV-এ লাগানো

- Hydrophone নিচের দিকে, frame থেকে কম্পন-বিচ্ছিন্ন। Speaker-এর জানালা খোলা পানির দিকে।
- Main computer-এর সাথে serial (15.3), অথবা শুধু BOOT button/LED দিয়ে।

19. Troubleshooting

লক্ষণ	সম্ভাব্য কারণ	কী করবে
Serial-এ "ADC start failed"	ভুল pin বা core 2.x	PIN_HYDROPHONE 32–39-এর একটা হতে হবে, core 3.x
preambles = 0, কিছুই আসে না	signal খুব কম, বা gain কম, বা ADC saturate	scope দেখো; stats-এ LEVEL; RV1/RV2 ঠিক করো; wire loopback দিয়ে DSP check
preambles বাড়ে কিন্তু crc_fail-ও বাড়ে	SNR সীমানায়, echo বেশি, বা clipping	q দেখো; clipped শূন্য রাখো; দূরত্ব কমিয়ে দেখো; hydrophone-এর অবস্থান বদলাও
clipped দ্রুত বাড়ে	gain বেশি বা thruster noise খুব জোরালো	RV2 কমাও; hydrophone thruster থেকে দূরে
overflows বাড়ে	CPU সময় পাচ্ছে না	loop()-এ লম্বা কাজ যোগ করোনি তো? WiFi/BT চালু করোনি তো?
fs_measured 49000–51500-এর বাইরে	ADC বা driver সমস্যা	board বদলে দেখো; issue খোলো
TX-এর সময় ESP32 reset	brownout: amp-এর current-এ voltage নামছে	আলাদা buck, 1000 µF, মোটা তার
Speaker গরম	অধ্যায় 7	checklist 7; MAX_TX_DUTY কমাও
Amp গরম/বন্ধ হয়ে যায়	speaker-এর তার GND-এ, বা 2 Ω-এর কম load	BTL নিয়ম (6.2); একটাই 4 Ω speaker
"HEAT-GUARD wait"	খুব ঘনঘন পাঠাচ্ছ	স্বাভাবিক, speaker রক্ষা করছে
LED জ্বলে কিন্তু "no reply"	অন্য দিকের TX বা RX কাজ করছে না	উল্টো দিকে ping; দুই ROV-এ ID আলাদা কি না দেখো
একই command দুইবার কাজ করে	seq বদলে নতুন command পাঠানো হয়েছে	স্বাভাবিক; repeat-এ seq একই থাকে, তখন দ্বিতীয়বার চলে না
Hum বা ভনভন শব্দ (scope-এ সব tone উঁচু)	ground loop, unshielded cable	star ground, shield শুধু এক প্রান্তে, preamp-এর supply filter

20. Safety

- LiPo battery: fuse ব্যবহার করবে, short হলে আগুন লাগতে পারে। পানিতে নামানোর আগে সব সংযোগ seal করবে।
- Pool-এ মানুষ থাকলে সাধারণত শব্দ নিরাপদ মাত্রায় থাকে (কয়েক W), কিন্তু খুব কাছ থেকে কানে পানির নিচে জোরালো tone অস্বস্তিকর। Test-এর সময় ডুবুরিদের জানিয়ে রাখো।
- Speaker আর amp গরম হলে বন্ধ করো। Box খোলার আগে ঠান্ডা হতে দাও।
- Epoxy হাতে লাগলে সাবান দিয়ে ধুয়ে ফেলবে। খোলা জায়গায় কাজ করবে।
- Competition-এর নিয়মে acoustic emission-এর কোনো সীমা থাকলে (pinger task-এর সময় চুপ থাকা ইত্যাদি) সেটা মেনে চলবে। দরকার হলে firmware-এ একটা "quiet" command যোগ করো।

A. সব formula এক জায়গায়

Formula	কোথায় লাগে	অধ্যায়
$\lambda = c / f$	wavelength, coating কতটা পাতলা	2.1, 9.1
$t = d / c$	শব্দ পৌঁছানোর সময়	2.1
$\Delta f = f \cdot v/c$	Doppler	2.4
$I = P/(\eta \cdot V)$	battery current	5
$f = 1/(2\pi\sqrt{(LC)})$	supply-র LC filter	5
$x(t) = V/2 + (2V/\pi)\Sigma\ \sin(n\omega t)/n$	square wave-এর ভেতরে কী আছে	6.2, 7.1
$f_c = 1/(2\pi RC)$	সব RC filter	6.2, 10.2
$R_{th} = R1 \parallel R2$, divider = $R2/(R1+R2)$	TX filter, volume	6.2
$P = V_{pk}^2/(2R)$	amp output power	6.2
$P_{DC} = (dV)^2/R$	DC-র অপচয়	7.1
$t_{rest} = t_{on}(1-d)/d$	heat guard	7.2
$\Delta T = P \cdot R_{th}$	তাপমাত্রা বৃদ্ধি	7.5
$\Delta T = (R_{hot}/R_{cold} - 1)/0.00393$	coil-এর তাপ মাপা	7.6
$P = \rho g h$	গভীরতায় চাপ	8.1
$w = Pa^4/(64D)$, $\sigma = 3Pa^2/(4t^2)$	জানালা কতটা বাঁকে	8.1
$Z = \rho c$, $T = 4Z_1Z_2/(Z_1+Z_2)^2$	seal করলে শব্দ ঢোকে কি না	9.1
$G = 1 + Rf/Rg$	op-amp gain	10.2
$GBW \geq G \cdot f$	op-amp বাছাই	10.2
$Fs > 2f$, $f_{alias} = \lvert f - kFs \rvert$	sampling, pinger alias	11.2
$PG = 10\log(B_{band}/B_{bin})$	processing gain	11.3
$f_k = 3000 + 250k$	tone	12.1
$\Delta f_{bin} = Fs/N$	frequency resolution	12.2
set = {s, s+6, s+12, s+18}	frequency hopping	12.4
$g(x) = x^{16}+x^{12}+x^5+1$	CRC-16	13.2
$G = (171, 133)_8$, $K = 7$	convolutional code	13.3
$slot = 37c \bmod 54$	interleaver	13.4
$dc \mathrel{+}= (x-dc)/1024$	DC blocker	14.1
$s[n] = xw + coef \cdot s[n-1] - s[n-2]$	Goertzel	14.2
$N \mathrel{+}= (\min(E,8N) - N)/256$	noise floor	14.3
score = mean $frac_j$	preamble detection	14.4

`err = (E_L−E_E)/(E_L+E_E)`	timing tracking	14.5
`Fs = ΔN/Δt`	ADC-র আসল rate	14.6
`L = max(ln z…) − max(ln z…)`	soft bit	14.7
`PM′ = max(PM ± L)`	Viterbi	14.8

B. শব্দকোষ (Glossary)

শব্দ	মানে
ADC	Analog-to-Digital Converter: voltage-কে সংখ্যায় রূপান্তর করে
DMA	Direct Memory Access: CPU ছাড়াই hardware নিজে data memory-তে রাখে
FSK	Frequency Shift Keying: তথ্য বোঝানো হয় কোন frequency বাজছে তা দিয়ে
Frequency hopping	সময়ের সাথে frequency-র set বদলানো
Slot / symbol	20 ms-এর একটা ভাগ, যাতে একটা tone বাজে
Preamble	Packet-এর শুরুর পরিচিত নকশা, যা দিয়ে packet খুঁজে পাওয়া যায়
Goertzel	একটা নির্দিষ্ট frequency-র শক্তি মাপার দ্রুত algorithm
Window (Hann)	Signal-এর দুই কিনারা নরম করার গুণক
SNR	Signal-to-Noise Ratio, dB-তে: $10 \cdot \log_{10}(P_{signal}/P_{noise})$
dB	অনুপাতের log মাপ। +3 dB ≈ ২ গুণ power, +10 dB = ১০ গুণ, +20 dB = ১০০ গুণ
Reverberation / echo	দেয়াল ও পানির উপরিতলে প্রতিফলিত শব্দ
Fading	Echo-গুলো একে অপরকে কেটে দিয়ে কোনো frequency দুর্বল হয়ে যাওয়া
CRC	ভুল ধরার check-সংখ্যা
Convolutional code / Viterbi	ভুল ঠিক করার code আর তার decoder
Interleaver	Bit-এর ক্রম এলোমেলো করে ছড়িয়ে দেওয়া
LLR / soft bit	Bit 0 না 1, আর কতটা নিশ্চিত
BTL	Bridge-Tied Load: speaker-এর দুই প্রান্তেই amp-এর output
Class-D	দ্রুত on/off switching amplifier, efficiency ~৯০%
Acoustic impedance	$Z = \rho c$, শব্দের জন্য পদার্থের "বাধা"
Aliasing	Fs/2-এর উপরের frequency-র ভুল করে নিচে দেখা দেওয়া
Buck converter	উচ্চ voltage থেকে দক্ষভাবে নিম্ন voltage বানানোর circuit
ACK	"পেয়েছি" উত্তর

C. পুরো source code

নিচের code repository-র file থেকে PDF বানানোর সময় সরাসরি বসানো হয়েছে, তাই হুবহু একই। নতুন version-এর জন্য সবসময় GitHub-এর code দেখবে।

firmware/DuburiModem/config.h

```
/*
 * config.h - the ONLY file you normally need to edit.
 *
 * Flash the same firmware on both vehicles. Give each one a different NODE_ID.
 */
#ifndef DUBURI_CONFIG_H
#define DUBURI_CONFIG_H

/* ---- Who am I ------------------------------------------------------- */
#define NODE_ID          1    /* 1..14, different on every vehicle            */
#define PEER_ID          2    /* default destination for commands             */

/* ---- Pins (ESP32 DevKit V1 / WROOM-32) ------------------------------- */
#define PIN_TX_AUDIO     25   /* square-wave tone out -> RC filter -> amp IN  */
#define PIN_HYDROPHONE   34   /* preamp output -> ADC1 channel 6 (input only) */
#define PIN_LED          2    /* on-board blue LED on most DevKit boards      */
#define PIN_BUTTON       0    /* BOOT button: press = send "toggle" to PEER   */
#define PIN_AMP_ENABLE   -1   /* amp SD/MUTE pin if your module has one, else -1 */
#define AMP_ENABLE_LEVEL HIGH /* level that turns the amp ON                  */

/* ---- Behaviour ------------------------------------------------------ */
#define USE_ACK          1    /* 1 = ask for a reply and retry if none comes  */
#define TX_RETRIES       3    /* attempts per command                         */
#define ACK_TIMEOUT_MS   2600 /* wait this long for a reply                   */
#define REPEAT_GAP_MS    400  /* gap between repeats when USE_ACK = 0         */
#define TURNAROUND_MS    300  /* receiver waits this long before replying     */
#define MUTE_TAIL_MS     120  /* keep RX deaf this long after our own TX      */
#define DEDUP_MS         15000/* same (src,seq) inside this time = repeat     */

/* ---- Heat protection -----------------------------------------------
 * Limits how much of the time the speaker may be playing. 0.5 means every
 * transmission must be followed by at least the same amount of silence
 * (a 1.16 s packet -> >= 1.16 s rest). Lower it if the speaker gets warm. */
#define MAX_TX_DUTY      0.5f

#define SERIAL_BAUD      115200

#endif
```

firmware/DuburiModem/DuburiModem.ino

```
/*
 * DuburiModem.ino - underwater acoustic link between two ROVs (ESP32).
 *
 * Board : "ESP32 Dev Module" (Arduino-ESP32 core 3.x, tested with 3.3.12)
 * Needs : no extra libraries.
 *
 * Same firmware on every vehicle; set NODE_ID in config.h.
 *
 * Serial Monitor (115200 baud, "Newline") commands:
 *   on [id]        turn LED on at vehicle id (default PEER_ID, 15 = all)
 *   off [id]       turn LED off
 *   toggle [id]    toggle LED
 *   ping [id]      ask for a reply, prints link quality
 *   send C A [id]  send command number C (0..255) with argument A (0..255)
 *   sweep          play every tone once (frequency response test)
 *   tone F MS      play F Hz for MS milliseconds
 *   scope          start/stop printing per-tone levels every 0.25 s
 *   stats          print counters, ADC level and measured sample rate
 *   help
 * Received packets are printed as:  RX src=2 dst=1 seq=3 cmd=1 arg=0 q=18.5dB
 * so a Jetson / Raspberry Pi can read the same serial port.
 */
#include <Arduino.h>
#include "config.h"
#include "am_core.h"
#include "esp_adc/adc_continuous.h"
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"

#if !defined(ESP_ARDUINO_VERSION_MAJOR) || ESP_ARDUINO_VERSION_MAJOR < 3
#error "Install Arduino-ESP32 core 3.x (Tools > Board > Boards Manager > esp32 by Espressif)"
#endif

/* Command numbers (you can add your own: 10..255 are free) */
enum {
  CMD_LED_ON = 1,
  CMD_LED_OFF = 2,
  CMD_LED_TOGGLE = 3,
  CMD_PING = 4,
};

/* ------------------------------------------------------------------------- */
/* Receiver task (core 0): ADC DMA -> modem core -> queue                    */
/* ------------------------------------------------------------------------- */
static am_rx_t g_rx;                       /* ~40 kB, keep it off the stack */
static SemaphoreHandle_t g_rx_lock;
static QueueHandle_t g_pkt_queue;
static adc_continuous_handle_t g_adc;
static adc_channel_t g_adc_chan;
static volatile uint32_t g_adc_overflows = 0;

/* measured ADC sample rate */
static volatile float g_fs_measured = AM_FS_NOMINAL;
/* ADC level (raw counts) over the last ~0.5 s */
static volatile float g_lvl_rms = 0, g_lvl_mean = 0;
static volatile uint16_t g_lvl_min = 0, g_lvl_max = 0;

static bool IRAM_ATTR onAdcOverflow(adc_continuous_handle_t, const adc_continuous_evt_data_t *, void *) {
  g_adc_overflows = g_adc_overflows + 1;
  return false;
}

static bool startAdc() {
  adc_unit_t unit;
  if (adc_continuous_io_to_channel(PIN_HYDROPHONE, &unit, &g_adc_chan) != ESP_OK || unit != ADC_UNIT_1) {
    Serial.println("ERROR: PIN_HYDROPHONE must be an ADC1 pin (32..39)");
    return false;
  }
  adc_continuous_handle_cfg_t hcfg = {};
```

```
  hcfg.max_store_buf_size = 16384;         /* 8192 samples = 164 ms of slack */
  hcfg.conv_frame_size = 1024;             /* 512 samples per read (~10 ms) */
  if (adc_continuous_new_handle(&hcfg, &g_adc) != ESP_OK) return false;

  adc_digi_pattern_config_t pat = {};
  pat.atten = ADC_ATTEN_DB_12;             /* ~0.15 .. 2.45 V usable */
  pat.channel = (uint8_t)g_adc_chan;
  pat.unit = ADC_UNIT_1;
  pat.bit_width = SOC_ADC_DIGI_MAX_BITWIDTH;

  adc_continuous_config_t cfg = {};
  cfg.pattern_num = 1;
  cfg.adc_pattern = &pat;
  cfg.sample_freq_hz = (uint32_t)AM_FS_NOMINAL;
  cfg.conv_mode = ADC_CONV_SINGLE_UNIT_1;
  cfg.format = ADC_DIGI_OUTPUT_FORMAT_TYPE1;
  if (adc_continuous_config(g_adc, &cfg) != ESP_OK) return false;

  adc_continuous_evt_cbs_t cbs = {};
  cbs.on_pool_ovf = onAdcOverflow;
  adc_continuous_register_event_callbacks(g_adc, &cbs, NULL);
  return adc_continuous_start(g_adc) == ESP_OK;
}

static void rxTask(void *) {
  static uint8_t raw[1024];
  static uint16_t smp[512];
  uint64_t total = 0;
  int64_t t_ref = 0;
  uint64_t n_ref = 0;
  int64_t t_last_fs = 0;
  /* level meter accumulators */
  double acc = 0, acc2 = 0;
  uint32_t accn = 0;
  uint16_t mn = 4095, mx = 0;

  for (;;) {
    uint32_t got = 0;
    esp_err_t r = adc_continuous_read(g_adc, raw, sizeof(raw), &got, 200);
    if (r != ESP_OK) continue;
    int64_t now = esp_timer_get_time();
    int n = 0;
    for (uint32_t i = 0; i + 1 < got; i += SOC_ADC_DIGI_RESULT_BYTES) {
      adc_digi_output_data_t *d = (adc_digi_output_data_t *)&raw[i];
      if (d->type1.channel != g_adc_chan) continue;
      uint16_t v = d->type1.data;
      smp[n++] = v;
      acc += v;
      acc2 += (double)v * v;
      if (v < mn) mn = v;
      if (v > mx) mx = v;
    }
    accn += n;
    total += n;

    xSemaphoreTake(g_rx_lock, portMAX_DELAY);
    am_rx_process(&g_rx, smp, n);
    am_rx_result_t res;
    while (am_rx_pop(&g_rx, &res)) xQueueSend(g_pkt_queue, &res, 0);
    xSemaphoreGive(g_rx_lock);

    /* Measure the real ADC rate against the CPU clock. The longer we run
     * the more exact it gets (DMA timing jitter / elapsed time). */
    if (t_ref == 0 && total > 25000) { t_ref = now; n_ref = total; t_last_fs = now; }
    if (t_ref != 0 && now - t_last_fs > 2000000) {
      t_last_fs = now;
      double fs = (double)(total - n_ref) * 1e6 / (double)(now - t_ref);
      if (fs > 20000.0 && fs < 62000.0) {
        g_fs_measured = (float)fs;
        xSemaphoreTake(g_rx_lock, portMAX_DELAY);
        if (fabsf(g_rx.fs - (float)fs) > 0.0002f * (float)fs) am_rx_set_fs(&g_rx, (float)fs);
        xSemaphoreGive(g_rx_lock);
      }
    }
    if (accn >= 25000) {
      double m = acc / accn;
```

```
      g_lvl_mean = (float)m;
      g_lvl_rms = (float)sqrt(fmax(acc2 / accn - m * m, 0.0));
      g_lvl_min = mn;
      g_lvl_max = mx;
      acc = acc2 = 0;
      accn = 0;
      mn = 4095;
      mx = 0;
    }
  }
}

/* ---------------------------------------------------------------------- */
/* Transmitter                                                            */
/* ---------------------------------------------------------------------- */
static int64_t g_tx_allowed_at = 0;   /* heat guard */
static uint8_t g_seq = 0;
static bool g_scope = false;
static bool g_led = false;

static void ampEnable(bool on) {
#if PIN_AMP_ENABLE >= 0
  digitalWrite(PIN_AMP_ENABLE, on ? AMP_ENABLE_LEVEL : !AMP_ENABLE_LEVEL);
  if (on) delay(20);
#else
  (void)on;
#endif
}

static void muteRx(uint32_t ms) {
  xSemaphoreTake(g_rx_lock, portMAX_DELAY);
  am_rx_mute(&g_rx, (uint32_t)((float)ms * g_fs_measured / 1000.0f));
  xSemaphoreGive(g_rx_lock);
}

static void waitUntil(int64_t t_us) {
  for (;;) {
    int64_t left = t_us - esp_timer_get_time();
    if (left <= 0) return;
    if (left > 3000) vTaskDelay(1);   /* sleep in 1 ms steps, spin the rest */
  }
}

/* Heat guard: after a transmission of T seconds the speaker must rest
 * T * (1 - duty) / duty seconds before the next one. */
static void heatGuard() {
  int64_t wait = g_tx_allowed_at - esp_timer_get_time();
  if (wait > 0) {
    Serial.printf("HEAT-GUARD wait %d ms\n", (int)(wait / 1000));
    waitUntil(g_tx_allowed_at);
  }
}

static void heatGuardUpdate(int64_t t_start, int64_t t_end) {
  float on = (float)(t_end - t_start);
  g_tx_allowed_at = t_end + (int64_t)(on * (1.0f - MAX_TX_DUTY) / MAX_TX_DUTY);
}

static void playTones(const uint8_t *tones, int count) {
  const int64_t slot_us = (int64_t)(AM_T_SLOT * 1e6f);
  heatGuard();
  uint32_t mute_ms = (uint32_t)(count * AM_T_SLOT * 1000.0f) + 40 + MUTE_TAIL_MS;
  muteRx(mute_ms);
  ampEnable(true);
  int64_t t0 = esp_timer_get_time() + 2000;
  waitUntil(t0);
  for (int i = 0; i < count; i++) {
    ledcWriteTone(PIN_TX_AUDIO, (uint32_t)lrintf(am_tone_hz(tones[i])));
    waitUntil(t0 + (int64_t)(i + 1) * slot_us);
  }
  ledcWrite(PIN_TX_AUDIO, 0);
  ampEnable(false);
  heatGuardUpdate(t0, esp_timer_get_time());
}

static void sendPacket(const am_packet_t &p) {
```

```
  uint8_t tones[AM_TOTAL_SLOTS];
  am_tx_build(&p, tones);
  playTones(tones, AM_TOTAL_SLOTS);
}

/* ---------------------------------------------------------------------- */
/* Packet handling                                                        */
/* ---------------------------------------------------------------------- */
static int16_t g_last_seq[16];
static uint32_t g_last_seq_ms[16];

static void setLed(bool on) {
  g_led = on;
  digitalWrite(PIN_LED, on ? HIGH : LOW);
}

static void sendAck(const am_packet_t &in) {
  am_packet_t a = {};
  a.dst = in.src;
  a.src = NODE_ID;
  a.seq = in.seq;
  a.flags = AM_FLAG_IS_ACK;
  a.cmd = in.cmd;
  a.arg = g_led ? 1 : 0;
  delay(TURNAROUND_MS);
  sendPacket(a);
  Serial.printf("TX ack to=%d seq=%d\n", a.dst, a.seq);
}

/* Returns true if this was the ACK we are waiting for. */
static bool handlePacket(const am_rx_result_t &r, int wait_src, int wait_seq) {
  const am_packet_t &p = r.pkt;
  if (p.dst != NODE_ID && p.dst != AM_ADDR_BROADCAST) {
    Serial.printf("RX (not for me) src=%d dst=%d q=%.1fdB\n", p.src, p.dst, r.q_db);
    return false;
  }
  if (p.flags & AM_FLAG_IS_ACK) {
    Serial.printf("ACK src=%d seq=%d cmd=%d state=%d q=%.1fdB\n", p.src, p.seq, p.cmd, p.arg, r.q_db);
    return (p.src == wait_src && p.seq == wait_seq);
  }
  Serial.printf("RX src=%d dst=%d seq=%d cmd=%d arg=%d flags=%X q=%.1fdB\n",
                p.src, p.dst, p.seq, p.cmd, p.arg, p.flags, r.q_db);

  uint32_t now = millis();
  bool repeat = (g_last_seq[p.src] == p.seq) && (now - g_last_seq_ms[p.src] < DEDUP_MS);
  g_last_seq[p.src] = p.seq;
  g_last_seq_ms[p.src] = now;
  if (!repeat) {
    switch (p.cmd) {
      case CMD_LED_ON: setLed(true); break;
      case CMD_LED_OFF: setLed(false); break;
      case CMD_LED_TOGGLE: setLed(!g_led); break;
      case CMD_PING: break;
      default: break; /* add your own commands here */
    }
  } else {
    Serial.println("   (repeat - action already done)");
  }
  /* Reply even to repeats: our earlier reply may have been lost. */
  if (USE_ACK && (p.flags & AM_FLAG_ACK_REQ) && p.dst == NODE_ID) sendAck(p);
  return false;
}

static void pollPackets() {
  am_rx_result_t r;
  while (xQueueReceive(g_pkt_queue, &r, 0) == pdTRUE) handlePacket(r, -1, -1);
}

static void sendCommand(int dst, uint8_t cmd, uint8_t arg) {
  am_packet_t p = {};
  p.dst = (uint8_t)dst;
  p.src = NODE_ID;
  p.seq = g_seq = (uint8_t)((g_seq + 1) & 0xF);
  p.cmd = cmd;
  p.arg = arg;
  bool want_ack = USE_ACK && dst != AM_ADDR_BROADCAST;
```

```
  if (want_ack) p.flags = AM_FLAG_ACK_REQ;

  for (int attempt = 1; attempt <= TX_RETRIES; attempt++) {
    Serial.printf("TX to=%d seq=%d cmd=%d arg=%d (try %d/%d)\n", dst, p.seq, cmd, arg, attempt, TX_RETRIES);
    sendPacket(p);
    if (!want_ack) {
      if (attempt < TX_RETRIES) delay(REPEAT_GAP_MS);
      continue;
    }
    uint32_t t0 = millis();
    while (millis() - t0 < ACK_TIMEOUT_MS) {
      am_rx_result_t r;
      if (xQueueReceive(g_pkt_queue, &r, pdMS_TO_TICKS(20)) == pdTRUE && handlePacket(r, dst, p.seq)) {
        Serial.printf("OK delivered to %d (try %d)\n", dst, attempt);
        return;
      }
    }
    Serial.println("   no reply");
  }
  if (want_ack) Serial.printf("FAIL no reply from %d\n", dst);
}

/* ------------------------------------------------------------------------ */
/* Diagnostics                                                              */
/* ------------------------------------------------------------------------ */
static void printStats() {
  xSemaphoreTake(g_rx_lock, portMAX_DELAY);
  am_stats_t s = g_rx.stats;
  float fs_used = g_rx.fs;
  xSemaphoreGive(g_rx_lock);
  Serial.printf("STATS node=%d preambles=%lu ok=%lu crc_fail=%lu last_q=%.1fdB timing_adj=%.1f\n",
                NODE_ID, (unsigned long)s.preambles, (unsigned long)s.packets_ok,
                (unsigned long)s.crc_fail, s.last_q_db, s.last_timing_adj);
  Serial.printf("ADC fs_measured=%.1f fs_used=%.1f overflows=%lu clipped=%lu\n",
                g_fs_measured, fs_used, (unsigned long)g_adc_overflows, (unsigned long)s.clipped);
  Serial.printf("LEVEL mean=%.0f rms=%.1f min=%u max=%u (want rms 20..300, min>100, max<3900)\n",
                g_lvl_mean, g_lvl_rms, g_lvl_min, g_lvl_max);
}

static void printScope() {
  float db[AM_NTONES];
  xSemaphoreTake(g_rx_lock, portMAX_DELAY);
  memcpy(db, g_rx.scope_db, sizeof(db));
  xSemaphoreGive(g_rx_lock);
  Serial.print("SCOPE");
  for (int i = 0; i < AM_NTONES; i++) Serial.printf(" %d", (int)lrintf(db[i]));
  Serial.printf("  rms=%.0f\n", g_lvl_rms);
}

static void doSweep() {
  Serial.println("SWEEP: each tone 300 ms. Watch 'scope' on the other vehicle.");
  for (int i = 0; i < AM_NTONES; i++) {
    uint8_t t[15];
    for (int k = 0; k < 15; k++) t[k] = (uint8_t)i;
    Serial.printf("SWEEP tone %d = %.0f Hz\n", i, am_tone_hz(i));
    playTones(t, 15);
    delay(150);
  }
}

static void doTone(int hz, int ms) {
  if (hz < 100 || hz > 20000 || ms < 1 || ms > 5000) {
    Serial.println("usage: tone <100..20000 Hz> <1..5000 ms>");
    return;
  }
  heatGuard();
  muteRx((uint32_t)ms + MUTE_TAIL_MS);
  ampEnable(true);
  int64_t t0 = esp_timer_get_time();
  uint32_t real = ledcWriteTone(PIN_TX_AUDIO, (uint32_t)hz);
  Serial.printf("TONE %d Hz (real %lu Hz) for %d ms\n", hz, (unsigned long)real, ms);
  delay(ms);
  ledcWrite(PIN_TX_AUDIO, 0);
  ampEnable(false);
  heatGuardUpdate(t0, esp_timer_get_time());
}
```

```
 377
 378  static void printHelp() {
 379    Serial.println("Commands: on|off|toggle|ping [id] ; send <cmd> <arg> [id] ; sweep ; tone <Hz> <ms> ; scope ;
stats ; help");
 380  }
 381
 382  static void handleLine(char *line) {
 383    char *w = strtok(line, " \t");
 384    if (!w) return;
 385    char *a1 = strtok(NULL, " \t"), *a2 = strtok(NULL, " \t"), *a3 = strtok(NULL, " \t");
 386    int id = a1 ? atoi(a1) : PEER_ID;
 387    if (!strcmp(w, "on")) sendCommand(id, CMD_LED_ON, 0);
 388    else if (!strcmp(w, "off")) sendCommand(id, CMD_LED_OFF, 0);
 389    else if (!strcmp(w, "toggle")) sendCommand(id, CMD_LED_TOGGLE, 0);
 390    else if (!strcmp(w, "ping")) sendCommand(id, CMD_PING, 0);
 391    else if (!strcmp(w, "send") && a1 && a2) sendCommand(a3 ? atoi(a3) : PEER_ID, (uint8_t)atoi(a1), (uint8_t)at
oi(a2));
 392    else if (!strcmp(w, "sweep")) doSweep();
 393    else if (!strcmp(w, "tone") && a1 && a2) doTone(atoi(a1), atoi(a2));
 394    else if (!strcmp(w, "scope")) g_scope = !g_scope;
 395    else if (!strcmp(w, "stats")) printStats();
 396    else printHelp();
 397  }
 398
 399  /* ------------------------------------------------------------------------ */
 400  void setup() {
 401    Serial.begin(SERIAL_BAUD);
 402    delay(300);
 403    pinMode(PIN_LED, OUTPUT);
 404    setLed(false);
 405    pinMode(PIN_BUTTON, INPUT_PULLUP);
 406  #if PIN_AMP_ENABLE >= 0
 407    pinMode(PIN_AMP_ENABLE, OUTPUT);
 408    ampEnable(false);
 409  #endif
 410    ledcAttach(PIN_TX_AUDIO, 5000, 10);
 411    ledcWrite(PIN_TX_AUDIO, 0);
 412    for (int i = 0; i < 16; i++) g_last_seq[i] = -1;
 413
 414    g_rx_lock = xSemaphoreCreateMutex();
 415    g_pkt_queue = xQueueCreate(8, sizeof(am_rx_result_t));
 416    am_rx_init(&g_rx, AM_FS_NOMINAL);
 417    if (!startAdc()) {
 418      Serial.println("ERROR: ADC start failed");
 419      for (;;) delay(1000);
 420    }
 421    /* core 0, above Arduino loop priority; it blocks on the ADC most of the time */
 422    xTaskCreatePinnedToCore(rxTask, "amrx", 6144, NULL, 5, NULL, 0);
 423
 424    Serial.printf("\nDUBURI modem node %d ready. Tones %.0f..%.0f Hz, packet %.2f s\n",
 425                  NODE_ID, am_tone_hz(0), am_tone_hz(AM_NTONES - 1), AM_TOTAL_SLOTS * AM_T_SLOT);
 426    printHelp();
 427  }
 428
 429  void loop() {
 430    static char line[64];
 431    static int len = 0;
 432    while (Serial.available()) {
 433      char c = (char)Serial.read();
 434      if (c == '\r') continue;
 435      if (c == '\n') {
 436        line[len] = 0;
 437        handleLine(line);
 438        len = 0;
 439      } else if (len < (int)sizeof(line) - 1) {
 440        line[len++] = c;
 441      }
 442    }
 443
 444    static uint32_t btn_ms = 0;
 445    if (digitalRead(PIN_BUTTON) == LOW && millis() - btn_ms > 500) {
 446      btn_ms = millis();
 447      sendCommand(PEER_ID, CMD_LED_TOGGLE, 0);
 448    }
 449
 450    static uint32_t scope_ms = 0;
```

```
  if (g_scope && millis() - scope_ms > 250) {
    scope_ms = millis();
    printScope();
  }
  pollPackets();
  delay(2);
}
```

firmware/DuburiModem/am_core.h

```
/*
 * am_core.h - DUBURI acoustic modem core (portable C99, no Arduino/ESP32 code).
 *
 * The same file is compiled into the ESP32 firmware AND into the PC test
 * harness (tests/ and sim/), so what we test on the PC is exactly what runs
 * on the vehicle.
 *
 * Signal design (see the PDF guide for the "why"):
 *   - 24 tones, 3000 Hz ... 8750 Hz, 250 Hz apart.
 *   - Time is cut into 20 ms "slots"; exactly one tone plays per slot.
 *   - Preamble: 10 slots, fixed tone pattern (used to find the packet start).
 *   - Data: 54 slots. Slot i uses hop-set (i % 6); each hop-set has 4 tones,
 *     so one slot carries 2 bits (4-FSK with frequency hopping).
 *   - 4 payload bytes + CRC-16 = 48 bits, + 6 zero "tail" bits = 54 bits,
 *     protected by a rate-1/2, K=7 convolutional code (the classic 171/133
 *     code used by Wi-Fi and space probes) -> 108 coded bits -> 54 slots.
 *   - The receiver uses soft-decision Viterbi decoding.
 *   - Coded bits are interleaved (spread in time and frequency) so that one
 *     faded tone or one noisy slot only damages scattered bits.
 */
#ifndef AM_CORE_H
#define AM_CORE_H

#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ---------------- Fixed signal parameters (TX and RX must match) -------- */
#define AM_FS_NOMINAL     50000.0f  /* RX ADC sample rate we ask for (Hz)   */
#define AM_T_SLOT         0.020f    /* one symbol slot (s)                  */
#define AM_T_WIN          0.016f    /* analysis window inside a slot (s)    */
#define AM_T_HOP          0.0025f   /* preamble search step (s)             */
#define AM_T_TRACK        0.001f    /* early/late offset for timing (s)     */

#define AM_F0             3000.0f   /* lowest tone (Hz)                     */
#define AM_DF             250.0f    /* tone spacing (Hz)                    */
#define AM_NTONES         24        /* tones 0..23 -> 3000..8750 Hz         */
#define AM_NSETS          6         /* frequency-hopping sets               */
#define AM_M              4         /* tones per set (2 bits per slot)      */

#define AM_PRE_LEN        10        /* preamble slots                       */
#define AM_PAYLOAD_BYTES  4
#define AM_FRAME_BYTES    6         /* payload + CRC16                      */
#define AM_K              7         /* convolutional code constraint length */
#define AM_INFO_BITS      (AM_FRAME_BYTES * 8 + AM_K - 1)  /* 54 incl. tail */
#define AM_CODED_BITS     (2 * AM_INFO_BITS)                /* 108          */
#define AM_DATA_SLOTS     (AM_CODED_BITS / 2)               /* 54           */
#define AM_TOTAL_SLOTS    (AM_PRE_LEN + AM_DATA_SLOTS)   /* 64 -> 1.28 s     */

/* ---------------- Receiver tuning (validated by sim/run_sim.py) --------- */
#ifndef AM_DET_SCORE_MIN
#define AM_DET_SCORE_MIN  0.14f     /* preamble score threshold (0..1)      */
#endif
#ifndef AM_DET_MATCH_MIN
#define AM_DET_MATCH_MIN  5         /* preamble slots whose tone must win   */
#endif
#ifndef AM_TRACK_ENABLE
#define AM_TRACK_ENABLE   1         /* early/late timing tracking on/off    */
#endif

/* ---------------- Buffers (sized for Fs up to ~60 kHz) ------------------ */
#define AM_RING           4096      /* power of two                         */
#define AM_W_MAX          1024
#define AM_HIST           128       /* preamble search history (hops)       */
#define AM_OUTQ           4         /* decoded packets waiting to be read   */

/* ---------------- Packet ------------------------------------------------ */
#define AM_ADDR_BROADCAST 15
#define AM_FLAG_ACK_REQ   0x8       /* sender wants an ACK back             */
#define AM_FLAG_IS_ACK    0x4       /* this packet IS an ACK                */
```

```
73
74  typedef struct {
75      uint8_t dst;    /* 0..15 (15 = everyone)              */
76      uint8_t src;    /* 0..15                              */
77      uint8_t seq;    /* 0..15, used to drop repeats        */
78      uint8_t flags;  /* 0..15, AM_FLAG_*                   */
79      uint8_t cmd;    /* 0..255, your command number        */
80      uint8_t arg;    /* 0..255, command argument           */
81  } am_packet_t;
82
83  /* ---------------- Coding helpers (used by TX and RX) -------------------- */
84  float    am_tone_hz(int tone_index);
85  uint16_t am_crc16(const uint8_t *data, int len);       /* CRC-16/CCITT-FALSE */
86  /* Convolutional code: n info bits -> 2n coded bits (encoder starts in state 0). */
87  void     am_conv_encode(const uint8_t *bits, int n, uint8_t *coded);
88  /* Soft Viterbi: llr[2n] (llr>0 means bit=0) -> n bits; assumes zero tail. */
89  void     am_viterbi(const float *llr, int n, uint8_t *bits);
90  /* Interleaver: coded bit c (0..107) -> stream position (2*slot + 0 MSB/1 LSB). */
91  int      am_interleave(int c);
92  void     am_pack(const am_packet_t *p, uint8_t frame[AM_FRAME_BYTES]);
93  int      am_unpack(const uint8_t frame[AM_FRAME_BYTES], am_packet_t *p); /* 1 = CRC ok */
94  extern const uint8_t am_preamble[AM_PRE_LEN];
95
96  /* ---------------- Transmitter ------------------------------------------
97   * Fills tones[] with the tone index (0..23) to play in each 20 ms slot.
98   * The firmware plays am_tone_hz(tones[i]) for slot i.
99   */
100 void am_tx_build(const am_packet_t *p, uint8_t tones[AM_TOTAL_SLOTS]);
101
102 /* ---------------- Receiver ---------------------------------------------- */
103 enum { AM_ST_SEARCH = 0, AM_ST_PEAK = 1, AM_ST_DECODE = 2 };
104
105 typedef struct {
106     uint32_t preambles;     /* preamble detections                        */
107     uint32_t packets_ok;    /* CRC passed                                 */
108     uint32_t crc_fail;      /* preamble found but CRC failed              */
109     uint32_t clipped;       /* ADC samples near 0 or 4095                 */
110     float    last_q_db;     /* link quality of last packet (dB)           */
111     float    last_timing_adj; /* samples moved by timing tracker          */
112 } am_stats_t;
113
114 typedef struct {
115     am_packet_t pkt;
116     float       q_db;       /* median per-slot SNR of the decided tones    */
117 } am_rx_result_t;
118
119 typedef struct {
120     /* configuration derived from the sample rate */
121     float    fs;
122     int      W, H, D;               /* window, hop, early/late offset      */
123     float    S;                     /* samples per slot (not an integer)   */
124     float    coef[AM_NTONES];       /* Goertzel 2cos(w)                    */
125     float    win[AM_W_MAX];         /* Hann window                         */
126     int      pre_off[AM_PRE_LEN];   /* hop offset of each preamble slot    */
127
128     /* input conditioning */
129     float    ring[AM_RING];
130     uint32_t n;                     /* total samples received              */
131     float    dc;
132     float    xw[AM_W_MAX];          /* scratch: windowed samples           */
133
134     /* noise floor per tone (for whitening) */
135     float    noise[AM_NTONES];
136     uint32_t noise_init;
137     float    scope_db[AM_NTONES];   /* last whitened levels (for debug)    */
138
139     /* preamble search */
140     int      state;
141     uint32_t next_hop;
142     uint32_t hop_count;
143     float    hist_frac[AM_HIST][AM_PRE_LEN];
144     uint8_t  hist_amax[AM_HIST];
145     uint32_t hist_end[AM_HIST];
146     float    pk_score[24];
147     uint32_t pk_end[24];
148     int      pk_n;
```

```
    float    pk_best;
    int      pk_below;

    /* decoding */
    uint32_t sync_base;             /* sample index of end of last preamble window */
    float    sync_frac;
    int      slot;
    float    llr[AM_CODED_BITS];
    float    slot_q[AM_DATA_SLOTS];
    float    track_adj;

    /* mute (while we transmit ourselves) */
    uint32_t mute_until;
    int      muted;

    /* output */
    am_rx_result_t outq[AM_OUTQ];
    int      out_head, out_count;
    am_stats_t stats;
} am_rx_t;

void am_rx_init(am_rx_t *rx, float fs);
/* Change the sample rate estimate (firmware measures the real ADC rate).
 * Ignored while a packet is being decoded; returns 1 if applied. */
int  am_rx_set_fs(am_rx_t *rx, float fs);
/* Feed raw 12-bit ADC samples (0..4095). */
void am_rx_process(am_rx_t *rx, const uint16_t *x, int count);
/* Ignore the next `samples` input samples (use while we transmit). */
void am_rx_mute(am_rx_t *rx, uint32_t samples);
/* Get one decoded packet; returns 1 if there was one. */
int  am_rx_pop(am_rx_t *rx, am_rx_result_t *out);
int  am_rx_state(const am_rx_t *rx);
void am_rx_get_stats(const am_rx_t *rx, am_stats_t *out);

/* For ctypes in the PC simulator */
int  am_rx_sizeof(void);

#ifdef __cplusplus
}
#endif
#endif /* AM_CORE_H */
```

firmware/DuburiModem/am_core.c

```
/*
 * am_core.c - DUBURI acoustic modem core. See am_core.h for the overview.
 *
 * Only single-precision float math is used (ESP32 has a float FPU but does
 * double in software, which is ~10x slower).
 */
#include "am_core.h"
#include <math.h>
#include <string.h>

#define AM_PI_F      3.14159265358979f
#define AM_RING_MASK (AM_RING - 1)
#define AM_PEAK_HOPS 14            /* hops collected after a preamble trigger */
#define AM_NOISE_INIT_HOPS 40      /* hops averaged for the first noise floor  */
#define AM_NOISE_ALPHA (1.0f / 256.0f)
#define AM_NOISE_CLIP  8.0f        /* a hop may raise the floor by at most x8  */
#define AM_LLR_CLAMP   20.0f

/* Preamble: 10 different tones, chosen so that
 *  1. neighbouring slots are far apart in frequency;
 *  2. data slots of another packet can never match more than 2 of the 10
 *     slots (their hop sets, tone % 6, do not line up);
 *  3. the echo of a preamble tone is >= 6 slots (120 ms) old before the same
 *     hop set is used by a data slot of this packet. */
const uint8_t am_preamble[AM_PRE_LEN] = {0, 14, 6, 22, 9, 17, 4, 21, 5, 11};

static int32_t sdiff(uint32_t a, uint32_t b) { return (int32_t)(a - b); }

float am_tone_hz(int tone_index)
{
    return AM_F0 + AM_DF * (float)tone_index;
}

/* CRC-16/CCITT-FALSE: poly 0x1021, init 0xFFFF. "123456789" -> 0x29B1 */
uint16_t am_crc16(const uint8_t *data, int len)
{
    uint16_t crc = 0xFFFF;
    for (int i = 0; i < len; i++) {
        crc ^= (uint16_t)data[i] << 8;
        for (int b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (uint16_t)((crc << 1) ^ 0x1021) : (uint16_t)(crc << 1);
    }
    return crc;
}

/* ---- Convolutional code, K=7, rate 1/2, generators 171/133 (octal) -------
 * Shift register reg = [newest input | 6 older inputs]. Every input bit gives
 * two output bits = parity of (reg AND generator). Free distance 10: two
 * different messages always differ in at least 10 coded bits. */
#define AM_G0 0x79   /* 171 octal = 1111001 */
#define AM_G1 0x5B   /* 133 octal = 1011011 */
#define AM_NSTATES 64

static int parity7(unsigned v)
{
    v ^= v >> 4;
    v ^= v >> 2;
    v ^= v >> 1;
    return (int)(v & 1u);
}

void am_conv_encode(const uint8_t *bits, int n, uint8_t *coded)
{
    unsigned state = 0;                         /* 6 previous inputs */
    for (int t = 0; t < n; t++) {
        unsigned reg = ((unsigned)(bits[t] & 1) << 6) | state;
        coded[2 * t] = (uint8_t)parity7(reg & AM_G0);
        coded[2 * t + 1] = (uint8_t)parity7(reg & AM_G1);
        state = reg >> 1;
    }
}

```

```c
/* Viterbi: keeps, for each of the 64 register states, the best-scoring path
 * so far (score = sum of +llr for expected 0 bits, -llr for expected 1 bits).
 * At the end the path that finishes in state 0 (because of the zero tail)
 * is traced back. */
void am_viterbi(const float *llr, int n, uint8_t *bits)
{
    uint64_t dec[AM_INFO_BITS];                /* chosen predecessor bit per state */
    float pm[AM_NSTATES], npm[AM_NSTATES];
    if (n > AM_INFO_BITS) n = AM_INFO_BITS;
    for (int s = 0; s < AM_NSTATES; s++) pm[s] = -1e30f;
    pm[0] = 0.0f;
    for (int t = 0; t < n; t++) {
        const float l0 = llr[2 * t], l1 = llr[2 * t + 1];
        uint64_t d = 0;
        for (int ns = 0; ns < AM_NSTATES; ns++) {
            unsigned u = (unsigned)ns >> 5;                 /* input bit that led here */
            float best = -1e30f;
            int bestb = 0;
            for (int b = 0; b < 2; b++) {
                unsigned ps = (((unsigned)ns & 31u) << 1) | (unsigned)b;
                unsigned reg = (u << 6) | ps;
                float m = pm[ps] + (parity7(reg & AM_G0) ? -l0 : l0) + (parity7(reg & AM_G1) ? -l1 : l1);
                if (m > best) { best = m; bestb = b; }
            }
            npm[ns] = best;
            if (bestb) d |= (uint64_t)1 << ns;
        }
        dec[t] = d;
        memcpy(pm, npm, sizeof(pm));
    }
    unsigned s = 0;                                         /* zero tail -> end in state 0 */
    for (int t = n - 1; t >= 0; t--) {
        bits[t] = (uint8_t)(s >> 5);
        unsigned b = (unsigned)((dec[t] >> s) & 1u);
        s = ((s & 31u) << 1) | b;
    }
}

void am_pack(const am_packet_t *p, uint8_t frame[AM_FRAME_BYTES])
{
    frame[0] = (uint8_t)(((p->dst & 0xF) << 4) | (p->src & 0xF));
    frame[1] = (uint8_t)(((p->seq & 0xF) << 4) | (p->flags & 0xF));
    frame[2] = p->cmd;
    frame[3] = p->arg;
    uint16_t crc = am_crc16(frame, AM_PAYLOAD_BYTES);
    frame[4] = (uint8_t)(crc >> 8);
    frame[5] = (uint8_t)(crc & 0xFF);
}

int am_unpack(const uint8_t frame[AM_FRAME_BYTES], am_packet_t *p)
{
    uint16_t crc = am_crc16(frame, AM_PAYLOAD_BYTES);
    p->dst = frame[0] >> 4;
    p->src = frame[0] & 0xF;
    p->seq = frame[1] >> 4;
    p->flags = frame[1] & 0xF;
    p->cmd = frame[2];
    p->arg = frame[3];
    return (frame[4] == (uint8_t)(crc >> 8)) && (frame[5] == (uint8_t)(crc & 0xFF));
}

/* Interleaver. Stream bits 2k (MSB) and 2k+1 (LSB) form slot k's 2-bit value;
 * slot k uses hop set k % 6. Coded bit c goes to slot (37*c) mod 54, MSB if
 * c < 54 else LSB. Neighbouring coded bits (which the Viterbi decoder needs
 * together) land >= 3 slots apart and in different hop sets, so one faded
 * frequency or one noise burst only produces scattered errors. */
int am_interleave(int c)
{
    int slot = (37 * c) % AM_DATA_SLOTS;
    return 2 * slot + (c >= AM_DATA_SLOTS ? 1 : 0);
}

static void frame_to_bits(const uint8_t frame[AM_FRAME_BYTES], uint8_t bits[AM_INFO_BITS])
{
    for (int i = 0; i < AM_FRAME_BYTES * 8; i++)
        bits[i] = (frame[i >> 3] >> (7 - (i & 7))) & 1;
```

```
    for (int i = AM_FRAME_BYTES * 8; i < AM_INFO_BITS; i++)
        bits[i] = 0;                                        /* tail */
}

void am_tx_build(const am_packet_t *p, uint8_t tones[AM_TOTAL_SLOTS])
{
    uint8_t frame[AM_FRAME_BYTES];
    uint8_t info[AM_INFO_BITS];
    uint8_t coded[AM_CODED_BITS];
    uint8_t stream[AM_CODED_BITS];

    am_pack(p, frame);
    frame_to_bits(frame, info);
    am_conv_encode(info, AM_INFO_BITS, coded);
    for (int c = 0; c < AM_CODED_BITS; c++)
        stream[am_interleave(c)] = coded[c];

    for (int i = 0; i < AM_PRE_LEN; i++)
        tones[i] = am_preamble[i];
    for (int k = 0; k < AM_DATA_SLOTS; k++) {
        int v = (stream[2 * k] << 1) | stream[2 * k + 1];
        tones[AM_PRE_LEN + k] = (uint8_t)((k % AM_NSETS) + AM_NSETS * v);
    }
}

/* ========================================================================== */
/*                               Receiver                                     */
/* ========================================================================== */

static void rx_configure(am_rx_t *rx, float fs)
{
    rx->fs = fs;
    rx->W = (int)lrintf(AM_T_WIN * fs);
    if (rx->W > AM_W_MAX) rx->W = AM_W_MAX;
    rx->H = (int)lrintf(AM_T_HOP * fs);
    rx->D = (int)lrintf(AM_T_TRACK * fs);
    rx->S = AM_T_SLOT * fs;
    for (int n = 0; n < rx->W; n++)
        rx->win[n] = 0.5f - 0.5f * cosf(2.0f * AM_PI_F * ((float)n + 0.5f) / (float)rx->W);
    for (int k = 0; k < AM_NTONES; k++)
        rx->coef[k] = 2.0f * cosf(2.0f * AM_PI_F * am_tone_hz(k) / fs);
    for (int j = 0; j < AM_PRE_LEN; j++)
        rx->pre_off[j] = (int)lrintf((float)(AM_PRE_LEN - 1 - j) * rx->S / (float)rx->H);
}

static void rx_reset_search(am_rx_t *rx)
{
    rx->state = AM_ST_SEARCH;
    rx->hop_count = 0;
    rx->pk_n = 0;
    rx->next_hop = rx->n + (uint32_t)rx->H;
}

void am_rx_init(am_rx_t *rx, float fs)
{
    memset(rx, 0, sizeof(*rx));
    rx_configure(rx, fs);
    rx->dc = -1.0f;                /* "not started": first sample sets it */
    rx_reset_search(rx);
}

int am_rx_set_fs(am_rx_t *rx, float fs)
{
    if (rx->state != AM_ST_SEARCH || fs < 20000.0f || fs > 62000.0f)
        return 0;
    rx_configure(rx, fs);
    rx_reset_search(rx);
    return 1;
}

int am_rx_sizeof(void) { return (int)sizeof(am_rx_t); }
int am_rx_state(const am_rx_t *rx) { return rx->state; }
void am_rx_get_stats(const am_rx_t *rx, am_stats_t *out) { *out = rx->stats; }

void am_rx_mute(am_rx_t *rx, uint32_t samples)
{
```

```
    rx->mute_until = rx->n + samples;
    rx->muted = 1;
    rx->state = AM_ST_SEARCH;       /* abandon anything half-received */
}

int am_rx_pop(am_rx_t *rx, am_rx_result_t *out)
{
    if (rx->out_count == 0) return 0;
    *out = rx->outq[rx->out_head];
    rx->out_head = (rx->out_head + 1) % AM_OUTQ;
    rx->out_count--;
    return 1;
}

/* Goertzel energy of `nt` tones over the W samples that end at sample `end`. */
static void rx_energies(am_rx_t *rx, uint32_t end, const uint8_t *tones, int nt, float *E)
{
    const int W = rx->W;
    uint32_t start = end - (uint32_t)W;
    for (int i = 0; i < W; i++)
        rx->xw[i] = rx->ring[(start + (uint32_t)i) & AM_RING_MASK] * rx->win[i];
    for (int t = 0; t < nt; t++) {
        const float c = rx->coef[tones[t]];
        float s1 = 0.0f, s2 = 0.0f;
        for (int i = 0; i < W; i++) {
            float s0 = rx->xw[i] + c * s1 - s2;
            s2 = s1;
            s1 = s0;
        }
        E[t] = s1 * s1 + s2 * s2 - c * s1 * s2 + 1e-3f;
    }
}

static const uint8_t all_tones[AM_NTONES] = {
    0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23};

static void rx_finish_peak(am_rx_t *rx);
static void rx_decode_slot(am_rx_t *rx);

static uint32_t rx_slot_end(const am_rx_t *rx, int slot)
{
    return rx->sync_base + (uint32_t)(int32_t)lrintf(rx->sync_frac + rx->track_adj + (float)(slot + 1) * rx->
S);
}

/* One preamble-search step: called every H samples. */
static void rx_hop(am_rx_t *rx)
{
    float E[AM_NTONES], z[AM_NTONES];
    rx_energies(rx, rx->n, all_tones, AM_NTONES, E);

    /* Noise floor per tone. Learned only while nothing is being received. */
    if (rx->state == AM_ST_SEARCH) {
        if (rx->noise_init < AM_NOISE_INIT_HOPS) {
            float k = 1.0f / (float)(rx->noise_init + 1);
            for (int i = 0; i < AM_NTONES; i++)
                rx->noise[i] += (E[i] - rx->noise[i]) * k;
            rx->noise_init++;
            rx->next_hop += (uint32_t)rx->H;
            return;
        }
        for (int i = 0; i < AM_NTONES; i++) {
            float e = E[i];
            if (e > AM_NOISE_CLIP * rx->noise[i]) e = AM_NOISE_CLIP * rx->noise[i];
            rx->noise[i] += (e - rx->noise[i]) * AM_NOISE_ALPHA;
        }
    }

    /* Whitening: each tone compared with its own noise floor. */
    float sum = 0.0f, zmax = 0.0f;
    int amax = 0;
    for (int i = 0; i < AM_NTONES; i++) {
        z[i] = E[i] / rx->noise[i];
        rx->scope_db[i] = 10.0f * log10f(z[i]);
        sum += z[i];
        if (z[i] > zmax) { zmax = z[i]; amax = i; }
```

```
    }

    int hi = (int)(rx->hop_count % AM_HIST);
    for (int j = 0; j < AM_PRE_LEN; j++)
        rx->hist_frac[hi][j] = z[am_preamble[j]] / sum;
    rx->hist_amax[hi] = (uint8_t)amax;
    rx->hist_end[hi] = rx->n;

    if (rx->hop_count >= (uint32_t)rx->pre_off[0]) {
        float score = 0.0f;
        int match = 0;
        for (int j = 0; j < AM_PRE_LEN; j++) {
            int h = (int)((rx->hop_count - (uint32_t)rx->pre_off[j]) % AM_HIST);
            score += rx->hist_frac[h][j];
            match += (rx->hist_amax[h] == am_preamble[j]);
        }
        score *= 1.0f / (float)AM_PRE_LEN;

        if (rx->state == AM_ST_SEARCH && score >= AM_DET_SCORE_MIN && match >= AM_DET_MATCH_MIN) {
            rx->state = AM_ST_PEAK;
            rx->pk_n = 0;
        }
        if (rx->state == AM_ST_PEAK) {
            rx->pk_score[rx->pk_n] = score;
            rx->pk_end[rx->pk_n] = rx->n;
            rx->pk_n++;
        }
    }
    rx->hop_count++;
    rx->next_hop += (uint32_t)rx->H;

    if (rx->state == AM_ST_PEAK && rx->pk_n >= AM_PEAK_HOPS)
        rx_finish_peak(rx);
}

/* The score is flat-topped around the true start; use the centre of the top. */
static void rx_finish_peak(am_rx_t *rx)
{
    int m = 0;
    for (int i = 1; i < rx->pk_n; i++)
        if (rx->pk_score[i] > rx->pk_score[m]) m = i;
    float top = rx->pk_score[m];
    float lim = 0.85f * top;
    int l = m, r = m;
    while (l > 0 && rx->pk_score[l - 1] >= lim) l--;
    while (r < rx->pk_n - 1 && rx->pk_score[r + 1] >= lim) r++;
    float base = 0.7f * top, wsum = 0.0f, isum = 0.0f;
    for (int i = l; i <= r; i++) {
        float w = rx->pk_score[i] - base;
        wsum += w;
        isum += w * (float)(i - l);
    }
    float c = (wsum > 0.0f) ? isum / wsum : 0.0f;

    rx->sync_base = rx->pk_end[l];
    rx->sync_frac = c * (float)rx->H;
    rx->track_adj = 0.0f;
    rx->slot = 0;
    rx->state = AM_ST_DECODE;
    rx->stats.preambles++;
    rx->next_hop = rx_slot_end(rx, 0) + (uint32_t)rx->D;
}

static float maxf(float a, float b) { return a > b ? a : b; }

static void rx_finish_packet(am_rx_t *rx)
{
    float cl[AM_CODED_BITS];
    uint8_t info[AM_INFO_BITS];
    uint8_t frame[AM_FRAME_BYTES] = {0};
    for (int c = 0; c < AM_CODED_BITS; c++)
        cl[c] = rx->llr[am_interleave(c)];
    am_viterbi(cl, AM_INFO_BITS, info);
    for (int i = 0; i < AM_FRAME_BYTES * 8; i++)
        frame[i >> 3] |= (uint8_t)(info[i] << (7 - (i & 7)));

```

```
    /* median of per-slot quality (insertion sort, 48 values) */
    float q[AM_DATA_SLOTS];
    memcpy(q, rx->slot_q, sizeof(q));
    for (int i = 1; i < AM_DATA_SLOTS; i++) {
        float v = q[i];
        int j = i - 1;
        while (j >= 0 && q[j] > v) { q[j + 1] = q[j]; j--; }
        q[j + 1] = v;
    }
    float qmed = 0.5f * (q[AM_DATA_SLOTS / 2 - 1] + q[AM_DATA_SLOTS / 2]);

    am_packet_t p;
    if (am_unpack(frame, &p)) {
        rx->stats.packets_ok++;
        rx->stats.last_q_db = qmed;
        rx->stats.last_timing_adj = rx->track_adj;
        if (rx->out_count < AM_OUTQ) {
            int idx = (rx->out_head + rx->out_count) % AM_OUTQ;
            rx->outq[idx].pkt = p;
            rx->outq[idx].q_db = qmed;
            rx->out_count++;
        }
    } else {
        rx->stats.crc_fail++;
    }
    rx_reset_search(rx);
}

static void rx_decode_slot(am_rx_t *rx)
{
    const int i = rx->slot;
    const int set = i % AM_NSETS;
    uint8_t tones[AM_M];
    float Ec[AM_M], Ee[AM_M], El[AM_M], z[AM_M], lz[AM_M];
    for (int v = 0; v < AM_M; v++)
        tones[v] = (uint8_t)(set + AM_NSETS * v);

    uint32_t e = rx_slot_end(rx, i);
    rx_energies(rx, e, tones, AM_M, Ec);
    int vhat = 0;
    for (int v = 0; v < AM_M; v++) {
        z[v] = Ec[v] / rx->noise[tones[v]];
        lz[v] = logf(z[v]);
        if (z[v] > z[vhat]) vhat = v;
    }

    float l1 = maxf(lz[0], lz[1]) - maxf(lz[2], lz[3]);  /* MSB: 0 for v=0,1 */
    float l0 = maxf(lz[0], lz[2]) - maxf(lz[1], lz[3]);  /* LSB: 0 for v=0,2 */
    if (l1 > AM_LLR_CLAMP) l1 = AM_LLR_CLAMP;
    if (l1 < -AM_LLR_CLAMP) l1 = -AM_LLR_CLAMP;
    if (l0 > AM_LLR_CLAMP) l0 = AM_LLR_CLAMP;
    if (l0 < -AM_LLR_CLAMP) l0 = -AM_LLR_CLAMP;
    rx->llr[2 * i] = l1;
    rx->llr[2 * i + 1] = l0;
    rx->slot_q[i] = 10.0f * log10f(maxf(z[vhat] - 1.0f, 1e-3f));

#if AM_TRACK_ENABLE
    /* Early/late gate: if the window shifted 1 ms later catches more of the
     * winning tone than the one shifted 1 ms earlier, we are a bit early. */
    if (z[vhat] > 4.0f) {
        uint8_t tv = tones[vhat];
        rx_energies(rx, e - (uint32_t)rx->D, &tv, 1, Ee);
        rx_energies(rx, e + (uint32_t)rx->D, &tv, 1, El);
        float err = (El[0] - Ee[0]) / (El[0] + Ee[0]);
        rx->track_adj += 0.25f * (float)rx->D * err;
        float lim = 0.25f * rx->S;
        if (rx->track_adj > lim) rx->track_adj = lim;
        if (rx->track_adj < -lim) rx->track_adj = -lim;
    }
#else
    (void)Ee;
    (void)El;
#endif

    rx->slot++;
    if (rx->slot >= AM_DATA_SLOTS)
```

```
        rx_finish_packet(rx);
    else
        rx->next_hop = rx_slot_end(rx, rx->slot) + (uint32_t)rx->D;
}

void am_rx_process(am_rx_t *rx, const uint16_t *x, int count)
{
    for (int k = 0; k < count; k++) {
        uint16_t raw = x[k];
        if (raw <= 8 || raw >= 4087) rx->stats.clipped++;
        float xf = (float)raw;
        if (rx->dc < 0.0f) rx->dc = xf;
        rx->dc += (xf - rx->dc) * (1.0f / 1024.0f);
        rx->ring[rx->n & AM_RING_MASK] = xf - rx->dc;
        rx->n++;

        if (rx->muted) {
            if (sdiff(rx->n, rx->mute_until) < 0) continue;
            rx->muted = 0;
            rx_reset_search(rx);
            continue;
        }
        while (sdiff(rx->n, rx->next_hop) >= 0) {
            if (rx->state == AM_ST_DECODE) rx_decode_slot(rx);
            else rx_hop(rx);
        }
    }
}

#ifdef AM_SIM
/* PC simulator only: copy out the soft bits and slot qualities of the last packet. */
void am_rx_debug(const am_rx_t *rx, float llr[AM_CODED_BITS], float slot_q[AM_DATA_SLOTS], float *track_adj)
{
    memcpy(llr, rx->llr, sizeof(rx->llr));
    memcpy(slot_q, rx->slot_q, sizeof(rx->slot_q));
    *track_adj = rx->track_adj;
}
#endif
```